

(Information) Security

- Keine unberechtigte Modifikation oder Extraktion von Informationen
- Schutz vor absichtlichen böswilligen Angriffen
- Reduktion von Schwachstellen
- z.B. Dos, Spam, Browsersnapping, Datenmanipulation

Safety

- System läuft und vermeidet Unfälle
- Schutz vor Unfällen menschlichen oder technischen Ursprungs
- Erkennung von Fehlern in der Funktionalität
- Spezifizierung der gewünschten Funktionalität und Erkennung von Abweichungen
- z.B. System-, Netzwerkversagen, Operating Errors

Sicherheitsziele CIA

- ① Confidentiality (Vertraulichkeit)
↳ Unbefugten Zugriff verhindern z.B. sensible Daten, die nicht jeder lesen darf
→ Maßnahmen: z.B. HTTPS, Verschlüsselung
- ② Integrity (Integrität)
↳ Unbefugte / unbeabsichtigte Änderungen verhindern z.B. manipulierte Daten
→ Maßnahmen: z.B. kryptografische Checksum / MAC oder Passwortwiederherstellung
- ③ Availability (Verfügbarkeit)
↳ Wenn benötigt: Zugriff auf Systeme/Daten/Funktionen ermöglichen z.B. DDoS-Angriffe (Zugriff auf Website blockieren)
→ Maßnahmen: z.B. Torpit
- ④ Authenticity (Authentizität)
↳ Echtheit von Daten / Akteuren beweisen (für wen + von wem sind die Daten?)
→ Maßnahmen: z.B. Nonces im Request, Zertifikate
- ⑤ Non-Repudiation (Nicht-Abstreikbarkeit)
↳ Jede Handlung nachweisen
→ Maßnahmen: z.B. digital Signatur

Definitionen

- ① Vulnerability (Schwachstelle)
= sicherheitsrelevante Schwäche in einem IT-System / Prozess z.B. veraltete Software mit bekannten Sicherheitslücken
- ② Threat (Bedrohung)
= Umstand / Ereignis, das Vulnerabilities nutzt, um Schäden zu verursachen z.B. Hackerangriff
 - Höhere Gewalt z.B. Naturkatastrophen
→ Backups
 - Technische Ausfälle z.B. Systemabstürze, HW- / SW-Defekte
→ Regelmäßige Inspektion + Backups
 - Organisatorische Mängel z.B. fehlende Richtlinien
→ Einführung eines ISMS mit Richtlinien
 - Menschliches Versagen z.B. Fehlkonfiguration, Phishing-Fallen
→ Schulung und Sensibilisierung der Mitarbeiter
 - Unachtsamkeit
 - Ignoranz
 - Bequemlichkeit
 - Zeit- oder Kostendruck
 - Absichtliche Angriffe z.B. Hacker, Malware
- ③ Risk (Risiko)
= Wahrscheinlichkeit mal Schadenshöhe
↳ Risiko reduzieren durch...
 - Passwort-Policies
 - Virenschutz + Firewalls
 - Notfallpläne
 - Backup-Konzepte
 - Verantwortungen vergeben
 - Verschlüsselung, Signaturen, usw.

Angriffe

Shell-Shock (2014)

- Grund: Programmierfehler im Parser beim Einlesen von Umgebungsvariablen
- Code Injection / Ausführung von beliebigem Code über Command Shell möglich

Locky Ransomware (2016)

- E-Mail-Anhänge mit Macros, die Malware installieren
- Malware verschlüsselt alle Dateien auf diesen und allen erreichbaren Geräten und löscht alle Shadow Copies z.B. im Cache
- Lösegeldzahlung vs. Löschung oder Veröffentlichung
- Schutzmaßnahmen:
 - offline Backup
 - keine unsensiblen Anhänge öffnen
 - Systeme updaten
 - immer mit minimalen Rechten arbeiten

Hardwarebasierte Probleme (2018)

- Grund: Fehlerhafte Computer/Prozessor-Architektur bei Intel-Prozessoren
 - Out-of-order-Ausführung, Spekulative Ausführung, gleiche PageTable für User-Prozesse und Kernel
- Maßnahmen:
 - Kernel-Page-Table-Isolation (KPTI)
 - Browser-Patches
 } Problem: Schlechtere Prozessor-Performance
- Meltdown
 - Absichtlich Exceptions herbeirufen, um auf den Speicher/Cache eines anderen Prozesses zuzugreifen
- Spectre
 - Scripting languages wie JavaScript greifen auf Address Space des Web-Browsers zu

Bordeaux-Hack (2015)

- Admin-Anmeldedaten klauen bzw. mit mimikatz-Tool per BruteForce herausfinden
- Diebstahl vieler sensiblen Daten

Drive-by-Download Attack

- User zwingen, bösartigen Code zu installieren

Pass-the-Hash (PtH) Attack

- Versucht gar nicht, das Passwort aus dem Hash zu rekonstruieren, sondern verwendet den Hash selbst
- ① Zugangsdaten herausfinden z.B. durch **Social Engineering** (= Leute mit psychologischen Tricks manipulieren, damit sie Dinge preisgeben, die geheim bleiben sollten) oder **Phishing**
- ② Lateral Movement → auf benachbarte Systeme im Netzwerk zugreifen (Zugriff auf gleicher Ebene ausweiten)
- ③ Privilege Escalation → höhere Zugriffsrechte (z.B. Admin-Rechte) beschaffen (Zugriff auf tieferen Ebenen)
- ④ Totale Kontrolle des Angreifers

Schutzmaßnahmen:

- Accounts und Accountrechte schützen und einschränken
- Firewalls
- Netzwerksegmentierung

Denial of Service (DoS) Attack

- Außergeplantsetzung von Netzwerken, Webseiten oder Services durch Überlastung über Fehlerprovokation oder Aufrufen zeitintensiver Operationen z.B. TCP SYN-Flooding (viele Requests, aber keine Acks beim TCP-Handshake)

Distributed Denial of Service (DDoS) Attack

- DoS-Angriff mit einer großen Menge an koordinierten Systemen

Replay-Attack

- Angreifer fängt legitime Datenpakete ab und sendet diese erneut, um z.B. unbefugten Zugriff auf Daten zu bekommen
- Maßnahmen: Zeitstempel, Nonce / Einmal-Token, TLS, MAC

Information Security Management System ISMS

① Warum schützen?

→ Security-Ziele setzen ⇒ CIA

② Was schützen?

→ Kritische Anwendungen und Daten identifizieren
→ Risikoanalyse

③ Wie schützen?

→ Aufgaben und Verantwortung verteilen
→ Policies und Richtlinien einführen
→ Awareness-Training für Mitarbeiter

z.B. PC sperren, wenn man Arbeitsplatz verlässt

④ Beweisen, dass geschützt:

→ Regelmäßige Audits durchführen lassen

⇒ kontinuierlichen Prozess!

- neue Gefahren ⇒ neue Maßnahmen nötig
- regelmäßige Reviews

Malware

Virus

- nur innerhalb eines Host-Programms
- kann sich nur verbreiten, wenn das infizierte Programm vom User geöffnet wird

Wurm

- eigenständiges Programm
- verbreitet sich eigenständig

Trojaner

- eigenständiges Programm, das sich als legitimes Programm tarnt
- verbreitet sich nicht, bleibt immer an selber Stelle
- keine Manipulation, sondern sammelt heimlich Daten wie Passwörter oder öffnet Backdoors für weitere Angriffe

Spyware

- sammelt sensible Daten

Ransomware

- verschlüsselt Daten und verlangt Lösegeld

Adware

- aggressive Werbesoftware, die Browserdaten sammelt

Bots

- Software, die automatisiert Tasks im Internet ausführt
- Botnets können so ganze Netzwerke übernehmen

Rootkit

- verschafft dem Angreifer Remote Access und Admin-Rechte, versteckt aber seine Gegenwart

Keylogger

- zeichnet Tastatureingaben und Daten in Zwischenablage auf und sendet sie an den Hacker

Exploit

- darüber können Angriffe auf Systeme mit Vulnerabilities durchgeführt werden

→ mehr dazu:
s. u.

Verschlüsselung

- dient der Confidentiality, also die Nachricht soll nur von bestimmten Personen gelesen werden können
- ⇒ Nur wer den richtigen Schlüssel hat, kann entschlüsseln

- Wandelt Plaintext mit einem Schlüssel zu einem Ciphertext um.
- Ciphertext kann mit dem richtigen Schlüssel wieder in Plaintext umgewandelt werden

Kerckhoffs Prinzip:

- = die Sicherheit eines kryptografischen Verfahrens hängt allein vom verwendeten Schlüssel ab und nicht von der Geheimhaltung des Algorithmus oder anderen Parametern

- ✓ **Open Design** → Sicherheit hängt nicht von Geheimhaltung des Designs / der Implementation ab
- ✗ **Security by obscurity** → System ist wie eine BlackBox, bei dem Teile des Prozesses geheim sind

außerdem: Niemals eigene Kryptografie verwenden! Immer bewährte Crypto Libraries hernehmen!

Reihenfolge

1. Compression
2. Verschlüsselung
3. Hashing / Checksum

Encrypt-then-MAC

digitale Signatur
s.u. aber:
Sign-then-Encrypt

vs. Encoding

- dient keinem CIA-Sicherheitsziel
- sondern dient dem Datentransfer und der Datenspeicherung
- ⇒ Jeder kann decodieren
- Repräsentiert Binärdaten als Ascii-Strings
- Teilt je 24 bit auf vermul. 6 bits auf und erweitert diese von 6 auf 8 bit
- ⇒ Nachricht wird um 33% länger!

vs.

Hashing

- dient der Integrität, also kann man Manipulationen erkennen → checksums
- ⇒ Irreversibel! Niemand kann es rückgängig machen
- Wandelt Plaintext in einen eindeutigen Hashwert / Digest um

Eigenschaften von (guten) Hashfunktionen

- Einwegfunktion
- Schnell und von jedem berechenbar
- keine Kollisionen
- gleicher Input → gleicher Output leicht veränderter Input → ganz anderer Output
- Output hat fixe Länge unabhängig von Input

z.B. SHA-2, SHA-3

Veraltet: MD5, SHA-1

Warum zuerst Compression, dann Encryption?

Weil die Komprimierung nur dann gut funktioniert, wenn sich viel wiederholt. Die Verschlüsselung bringt das durcheinander, weil die Daten dann durcheinandergewürfelt werden → Entropie steigt

Wozu überhaupt Compression?

- verschleunigt die Verschlüsselung
- erschwert Kryptoanalyse, die versucht, über Wiederholungen / Redundanzen im Plaintext Muster zu erkennen

Warum zuerst Encryption, dann Error Detection?

Weil wenn ich nur den Plaintext für die Checksum verwende, kann ich nicht feststellen, ob der Ciphertext manipuliert wurde!

Wozu überhaupt Checksum?

- um Manipulationen im Ciphertext zu erkennen

Was muss ich noch beachten?

- Kryptografische Software fällt unter Waffengesetz → Patente und Restriktionen beim Export
- Ich kann Encryptions speichern:

XML Encryption Standard

- ganzes XML-Dokument oder einzelne (Sub-)Elemente
- Multi-recipient encryption für mehrere Empfänger
- von vielen Libraries unterstützt
- Element in XML-Struktur:
 - EncryptedData (umschließender Tag)
 - EncryptionMethod (die verwendete Methode)
 - CipherData (Daten direkt oder als Referenz)
 - CipherValue oder • CipherReference

PKCS#7 Enveloped Data

- anderes Format, um Verschlüsselungen zu speichern
 - Syntax
- ```
EnvelopedData ::= SEQUENCE { ... }
```

## Angriffsmöglichkeiten auf Verschlüsselungen

- Brute Force
- Kryptoanalyse (mathematische Analyse) ⇒ Schlüssel herausfinden
  - Known Ciphertext      - Known Plaintext
  - Chosen Ciphertext    - Chosen Plaintext
- Social Engineering
- Vulnerabilities im System suchen, z.B. Masterkey liegt noch der Key im Cache!
- Seitenkanalangriffe → physikalische Größen, z.B. Energieverbrauch, Elektromagnetische Strahlung, Zeit für eine Operation
- Fault Attack → aktiv falsche kryptografische Operationen verwenden / durchführen
- Man-in-the-Middle-Angriff

## Institutionen → Vorgaben + Empfehlungen

- BSI - Bundesamt für Sicherheit in der Informationstechnik
- NIST - National Institute of Standards and Technology
- NSA - National Security Agency
- TÜV IT - naja TÜV halt für IT

### Verschlüsselung Encoding Hashing

nur wer Schlüssel hat, kann entschlüsseln  
jeder kann decodieren  
geht nur in eine Richtung



## Symmetrische Verschlüsselung

- gleicher Schlüssel für Ver- und Entschlüsselung
- ⊕ **Schnell**, in HW implementierbar
- ⊖ **Schlüsselaustauschproblem**
- z.B. Block Ciphers (→ DES, AES)
- Stream Ciphers (→ RC4, ChaCha20)

vs.

## Asymmetrische Verschlüsselung

- Schlüsselpaar: Public key und private key
- ⊖ **hoher Rechenaufwand**
- ⊕ **Sicherer Schlüsselaustausch**
- z.B. RSA, ECIES Elliptische Kurven, Diskreter Logarithmus

## Hybride Verschlüsselung

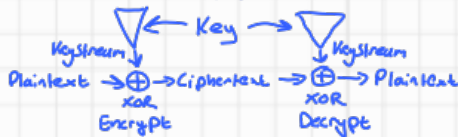
- Daten werden symmetrisch verschlüsselt
  - Schlüsselmanagement erfolgt asymmetrisch
  - z.B. ECIES
- best of both worlds
- ⊕ schnelle Datenver- und -entschlüsselung
  - ⊕ sicheren Schlüsselaustausch
- Symmetrischer Session Key wird asymmetrisch verschlüsselt/ausgetauscht

## Stream Ciphers

- Fixe Schlüssellängen, meist 256 oder 512 bit
- Erzeugt Datenstrom beliebigen Länge, der aus pseudo-zufälligen Zahlen besteht
- Jedes Bit (oder Byte) wird einzeln ge-xor-t (sehr schnelle Operation)

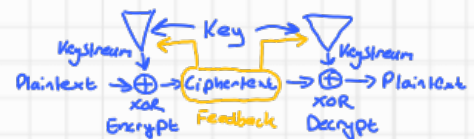
### Synchron

- Key Stream wird unabhängig vom Klartext oder Key Text generiert



### asynchron

- Key Stream hängt vom zuvor verschlüsselten Bit ab



- ⊕ Sicherer, da anderer Plaintext auch andere Keystreams erzeugt
- ⊖ Langsamer, kann den ganzen Key Stream aufhalten

## Stream Ciphers

## Block Ciphers

- Verarbeitet den Klartext und Geheimtext blockweise, z.B. 64 oder 128 bit Blocklänge
- Es gibt verschiedene Block Cipher Modes
- Können als Stream Ciphers implementiert werden

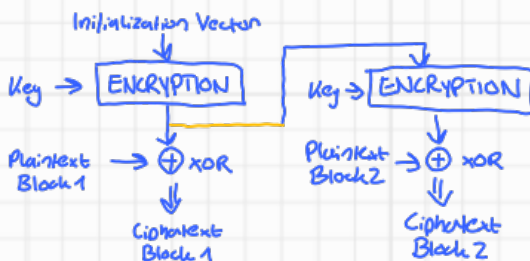
## Padding

- den letzten, unvollständigen Block mit unregelmäßigen, aber rekonstruierbaren Mustern auffüllen
- braucht man u.a. für viele Block-Chiffren, die vollständige Blöcke benötigen
- z.B. PKCS5

## Block Cipher Modes

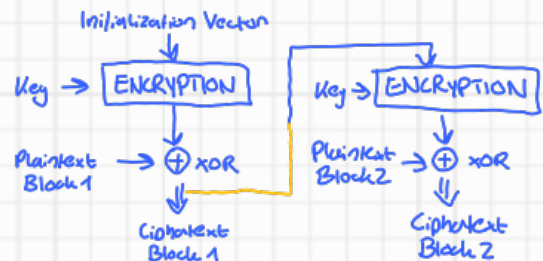
### Output Feedback Mode (OFB)

- Synchrones Stream Ciphering (Key Stream unabhängig vom Plaintext)
- Key Stream kann vorher ausgerechnet werden



### Cipher Feedback Mode (CFB)

- Selbst-synchronisierendes Stream Ciphering (Key Stream hängt vom Ciphertext ab)
- Fehler im IV oder Ciphertext werden nach 2 Blöcken gelöst

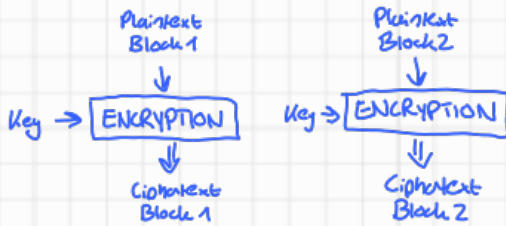


↑ Schlecht

↑ besser

## Electronic Code Book (ECB)

- Jeder Plaintext-Block wird unabhängig verschlüsselt
- ⊖ Gleicher Plaintext-Block führt immer zu gleichem Ciphertext-Block
- ⊖ Stereotypischer Beginn + Ende → leicht über Kryptanalyse zu knacken
- ⊕ Blöcke können unabhängig + parallel verarbeitet werden
- ⊕ Bitfehler in einem Block haben keine Auswirkungen auf die anderen
- nach Bit-Verlust müssen die Block-Boundaries neu synchronisiert werden

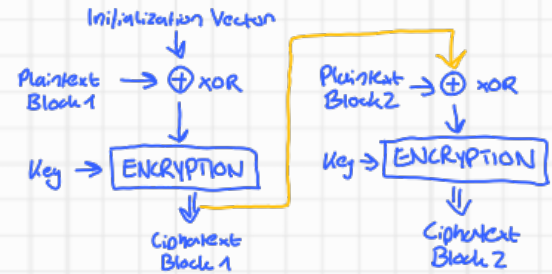


## Cipher Block Chaining (CBC)

- Feedback: Verschlüsselung eines Blocks hängt vom vorherigen Block ab
- Plaintext wird vor der Verschlüsselung mit vorherigem Ciphertext ge-XOR-t (bzw. IV beim ersten Block)
- ⊕ sicherer, schwerer zu knacken
- ⊖ Error Propagation: Bitfehler werden erst nach 2 Blöcken gelöst
- ⊖ Keine Recovery nach Synchronisationsfehler möglich (also wenn ein Bit verloren geht)

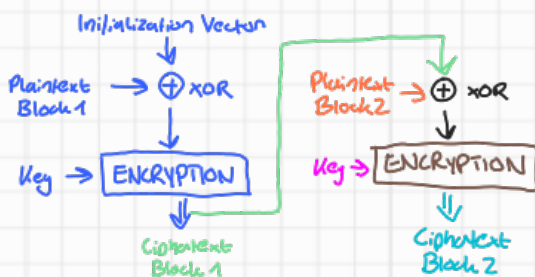
### Mögliche Angriffe:

- Padding Oracle Attack
- Angreifer können Ciphertext bearbeiten und dadurch den Plaintext modifizieren!



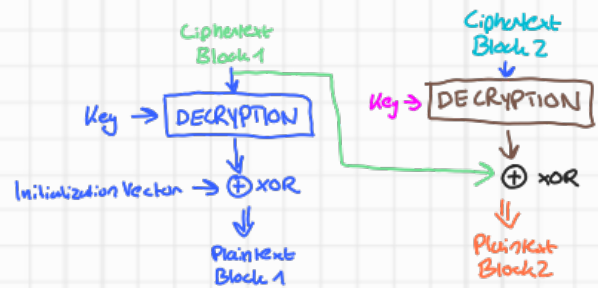
## Verschlüsselung:

$$C_i = E_k(P_i \oplus C_{i-1})$$



## Entschlüsselung

$$P_i = C_{i-1} \oplus D_k(C_i)$$

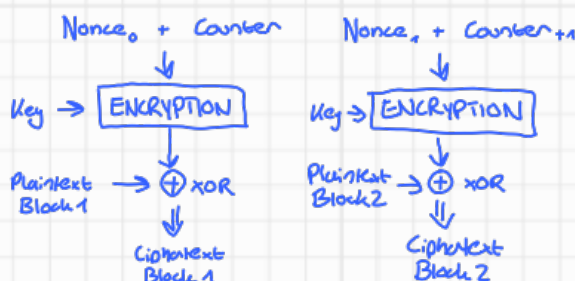


## Counter Block Mode (CTR)

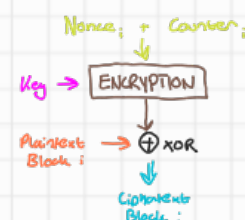
- Für jeden Block wird ein neuer, unvorhersagbarer Keystream erzeugt
- Keystream Block = IV (nonce) + current counter + encryption key
- ⊕ Key hängt nicht von Key aus vorherigen Block ab
- ⊕ Ver- und Entschlüsselung können parallel erfolgen
- ⊕ Keystream kann vorher ausgerechnet werden, wenn man es als Stream Cipher verwendet
- ⊕ Bitfehler haben keine Auswirkungen auf andere Blöcke

### Nonce

= number used only once (random)



$$C_i = P_i \oplus E_k(ctr_i)$$



→ der beste Mode

## Galois Counter Mode (GCM)

- kann man als Verschlüsselung + Hash, aber auch als standalone MAC verwenden
- erzeugt einen AuthTag, der enthält:
  - Authentifizierungsdaten
  - den IV und
  - den Ciphertext

⊕ schnell, parallele Verarbeitung mögl.

einer der wenigen BlockCipher Modes, der AEAD unterstützt

Confidentiality  
+  
Integrity  
+  
Authenticity

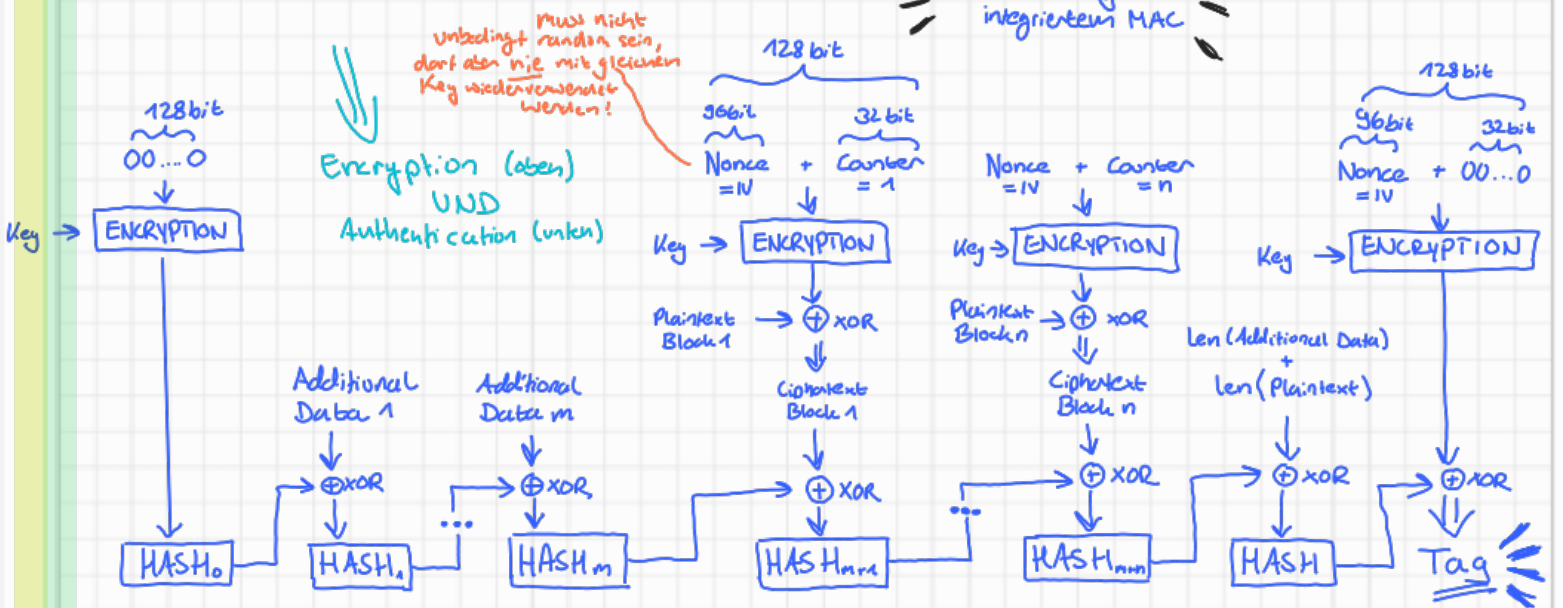
weil Verschlüsselung  
weil MAC  
→ sicher gegen Manipulation des Ciphertexts

## Authenticated Encryption mode with Associated Data (AEAD)

↳ d.h. Daten werden verschlüsselt und gecheckt

↳ d.h. man kann optional auch unverschlüsselte Daten / Plaintext hashen

= Verschlüsselung mit integriertem MAC



## Initialization Vector (IV)

- ist nicht geheim und kann unverschlüsselt versandt werden
- sollte zufällig sein
- sollte bei jeder Message anders sein
- hat die gleiche Größe wie die Block-Size, bei AES immer 128 bit

- niemals den Key als IV verwenden!  
(sonst kann man den Key evtl. lesen)
- niemals auf einen konstanten Wert wie 0 setzen!  
(IV soll Randomness reinbringen)
- niemals IV aus einem Passwort generieren!  
(das Passwort müsste bei jedem Nachricht geändert werden)

## Block Cipher Modes

## Advanced Encryption Standard (AES)

- Block-Chiffre
- Leicht in HW und SW implementierbar
- Nachfolger vom Data Encryption Standard (DES), welcher heute als unsicher gilt
- Basiert auf dem Rijndael-Algorithmus
- Block-Size: 128 bit (16 Bytes)
- Key-length: 256 bit

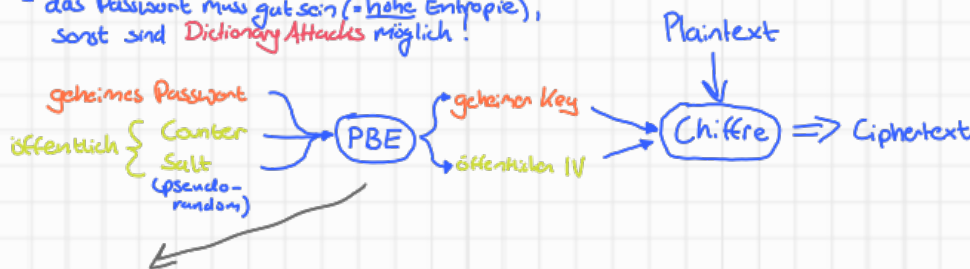
Verwendet gerne GCM als Modus  
⇒ AES-GCM

## Wie funktioniert AES?

- Open-design: kein Geheimnis, öffentlich einsehbar
- es gibt mehrere Runden, in denen der Plaintext z.B. mit Matrizen ge-xor-t wird

## Password Based Encryption (PBE)

- generiert symmetrische Keys aus Passwörtern
- das Passwort muss gut sein (= hohe Entropie), sonst sind Dictionary Attacks möglich



für die Entschlüsselung brauche ich den Key, den IV, das Salt und den Counter

## Standards:

- PKCS#5 verwendet die Password-Based Key Derivation Function 2 (PBKDF2), um aus dem Passwort einen Key zu machen
- PKCS#12 definiert das Dateiformat, in dem die private Keys inkl. zugehörigem Zertifikat gespeichert werden



## Verschlüsselung (Confidentiality)

### ChaCha20

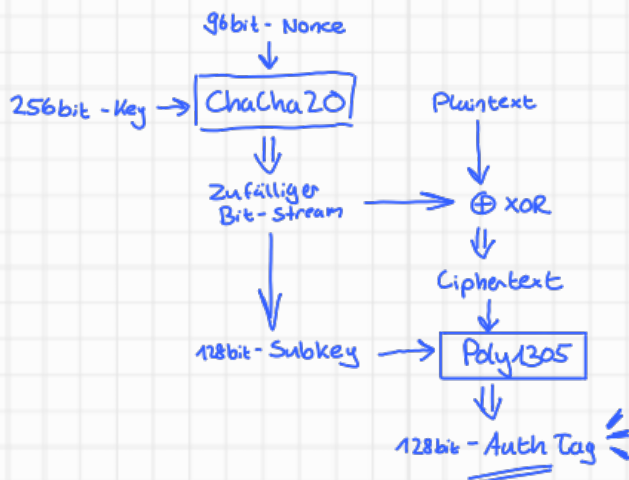
- Symmetrische Stream Cipher
- basiert auf Salsa20 von Daniel Bernstein

- leichter zu implementieren als AES-GCM
- verwendet in IPSec, TLS, OpenSSH, OpenSSH

## Hash (Integrity)

### Poly1305

- sehr schneller MAC



## Symmetrische Verschlüsselung

### Rivest-Shamir-Adleman RSA

- Asymmetrisches Verschlüsselungsverfahren
- leicht zu verstehen und zu implementieren, lizenzfrei:  $\rightarrow$  sehr beliebt
- Grundlage: Primfaktorzerlegung von großen Zahlen
- Key Length: zwischen 2000 und 3000
- sehr viel langsamer als AES

1. Finde zwei sehr große Primzahlen  $p, q$
2. Berechne Modul  $n = p \cdot q$
3. Wähle ein  $e < n$  mit  $\text{ggT}(e, (p-1) \cdot (q-1)) = 1$
4. Wähle ein  $d$  mit  $e \cdot d \bmod (p-1) \cdot (q-1) = 1$
5. Public key:  $(e, n)$  Private key:  $(d, n)$

$\rightarrow$  verschlüsseln mit  
 $\text{Ciphertext} = \text{plaintext}^e \bmod n$

$\rightarrow$  entschlüsseln mit  
 $\text{plaintext} = \text{ciphertext}^d \bmod n$

### Elliptic Curve Cryptography (ECC)

- + schneller als RSA, da deutlich kürzere Key length nötig
- + sicher bei Quantencomputern
- 1. Gerade durch zwei Punkte auf Kurve legen
- 2. Nimm gespiegelten Punkt des dritten Schnittpunkts
- 3. Gerade durch Punkt A und Spiegelpunkt usw. usw.
- $\rightarrow$  Public key: Start- und Endpunkt
- Private key: Anzahl an durchlaufenen Runden



z.B.  
 $y^2 = x^3 - 4x^2 + 10$

## Asymmetrische Verschlüsselung

### Elliptic Curve Integrated Encryption Scheme (ECIES)

- kombiniert ECC + symmetrische Chiffre + MAC
- kein konkreter Algorithmus, sondern ein Framework
- z.B. AES-CTR, AES-GCM, ChaCha20-Poly1305

1. Empfänger besitzt einen public Key  $V$ . Sender erzeugt kurzlebigen public Key  $U$  und kurzlebigen private Key  $u$ .
2. Asymmetrische Schlüsselvereinbarung, z.B. ECDH (Elliptic Curve Diffie-Hellman)  $\rightarrow u \cdot V$
3. Schlüsselgenerierung durch eine Key Derivation Function, z.B. eine kryptografische Hashfunktion  $\rightarrow$  zwei Keys: einen für MAC  $k_{\text{mac}}$ , einen fürs Verschlüsseln  $k_{\text{enc}}$ .
4. Symmetrische Verschlüsselung mit  $k_{\text{enc}}$ , z.B. mit AES-GCM oder ChaCha20
5. Checksum bzw. AuthTag berechnen mit  $k_{\text{mac}}$  und dem Ciphertext

fürs Entschlüsseln braucht man dann  $U$  und  $v$

## Hybride Verschlüsselung

# Key Management

- Generierung
  - Speicherung
  - Zurückziehen
  - Übermittlung
  - Backups
  - Zerstörung
- von Keys

## Schlüsselgenerierung

- gute random Numbers nötig!
- an einem sicheren Ort, z.B. smart card, HSM

## Schlüsselenkung → lock-exchange-reencrypt

1. Wenn z.B. jmd. deinen Key herausgefunden hat, kann man den sperren (wie eine verlorene Bankkarte)
2. Schlüssel durch neuen ersetzen
3. Daten neu verschlüsseln (oft nicht so leicht)

→ auch: wenn Key nicht mehr sicher ist, z.B. wegen zu kurzer Schlüssellänge

## Übermittlung von Keys

- basiert meist auf asymmetrischen Verfahren, z.B. DH, ECDH, RSA, PKI

## Diffie-Hellman-Schlüsselvereinbarung (DH)

- nicht geeignet fürs Verschlüsseln
- Beide Parteien berechnen den gemeinsamen, geheimen Session Key für sich selbst  
→ Schlüssel wird nie übers Netzwerk übertragen
- Schlüssel wird dann für symmetrische Verfahren verwendet
- Basiert auf dem Problem des diskreten Logarithmus
- Schlüssellängen: 2000 bis 3000 bit

1. Öffentliche Parameter  $n$  und  $g$
2. Alice wählt privates  $x$  und berechnet  $X = g^x \bmod n$   
Bob wählt privates  $y$  und berechnet  $Y = g^y \bmod n$
3.  $X$  und  $Y$  sind öffentlich und werden übertragen
4. Alice berechnet privaten Schlüssel  $k = Y^x \bmod n$   
Bob berechnet privaten Schlüssel  $k = X^y \bmod n$

## Zufallszahlen

- Qualität der Verschlüsselung hängt von Qualität der Keys ab
- Entropie als Maß der Unordnung / Randomness → maximale Entropie ⇒ gleichmäßige Verteilung
- je höher die Entropie, desto schwieriger sind Brute Force Attacks
- Problem: Echte Zufälligkeit schwer zu bekommen, da Computer deterministische Maschinen sind
- Lösung: so viele schwer vorauszusagende Quellen nutzen wie möglich, z.B.: Interrupts, User Input, Noise von Sound Card, ...

## Eigenschaften guter RNGs

- generiert gleichmäßig verteilte Zahlen
- Zahlen können nicht vorhergesagt werden
- haben einen langen und vollständigen Zyklus, d.h. keine Wiederholungen, bevor nicht alle anderen möglichen Werte durchgekommen sind

Hinweis zur Verwendung von z.B.

SecureRandom random = new SecureRandom();

im Code: nicht jedes Mal neue Instanz erzeugen, sondern diese weiterverwenden

## Speicherung von Schlüsseln / Anmeldedaten / Tokens

- Niemals im Source Code
- in Config-Files / als Umgebungsvariablen nur verschlüsselt oder mit strenger Access Control
- Password Based Encryption (PBE)
- externe Key-Stores
  - Vaults / Secret Manager
  - Sealed Secrets in git

als Service mit Zugriff per API mit Authentikation und Authorization  
in einem Hardware Security Module (HSM)
- In Hardware / Chip Card + PIN
- Key Splitting → Schlüssel in  $n$  Teile teilen, wovon  $m$  gebraucht werden, um Key zu rekonstruieren



## Zerstörung von Schlüsseln

- Dateien und Speicherbereich auf Hard Disk überschreiben
- Cache löschen
- ggf. HW zerstören

## Backups von Schlüsseln

- für Ausnahmesituationen, z.B. Tod, Urlaub, jmd. hint auf, dort zu arbeiten
- Verlust eines Keys → Key Recovery
- Law Enforcement → Key Escrow
- Secret Sharing (ähnlich wie Key Splitting)
- Master Key
- Niemals Backups/Kopien von Private Signature Keys machen!

- sonst könnten Signaturen gefälscht werden  
- die würde man dann nämlich einfach neu generieren

## Elliptic Curve Diffie-Hellman (ECDH)

- deutlich kürzere Schlüssellängen
- 1. Wähle sichere öffentliche Parameter
  - Primzahl  $p$
  - Kurvenparameter  $a, b$
  - Punkt  $P$  auf Kurve
  - Ordnung  $q$  von  $P$
  - Kofaktor  $i$
- 2. Alice wählt gleichverteiltes zufälliges privates  $x \in \{1, \dots, q-1\}$   
Bob wählt gleichverteiltes zufälliges privates  $y \in \{1, \dots, q-1\}$
- 3. Alice berechnet und sendet öffentliches  $Q_A = x \cdot P$   
Bob berechnet und sendet öffentliches  $Q_B = y \cdot P$
- 4. Alice berechnet gemeinsamen privaten Schlüssel  $k = x \cdot Q_B = x \cdot y \cdot P$   
Bob berechnet gemeinsamen privaten Schlüssel  $k = y \cdot Q_A = x \cdot y \cdot P$

True RNGs

- HW-basiert
- komplex und teuer

PRNGs (Pseudo Random Number Generators)

- SW-basiert
- schnell, aber vorhersagbar

↓ Lösung: Mix

- Verwende externe Quellen wie z.B.
  - Uhrzeit
  - CPU-interne Counter
  - Tastatureingaben
  - Maus-Bewegungen
- als initial Value und berechne die Zufallszahlen progressiv



# Integrität

- Keiner kann heimlich meine Daten manipulieren, ohne dass ich's mitfühle
- Viele Daten  $\rightarrow$  kurze Checksum zur Überprüfung
- 3 Levels:
  - Hashing  $\rightarrow$  NICHT sicher gegen Tampering (= Man-in-the-Middle Attacks, bei denen die Daten manipuliert werden und Empfänger meint es nicht)
  - Kryptografisches Hashing  $\rightarrow$  MAC (= Checksum + Secret)
  - Digitale Signaturen

## Kryptografisch starke Hash-Funktionen

- SHA-256
- SHA-384
- SHA-512
- SHA3-256
- SHA3-384
- SHA3-512

unsicher: MD5, SHA-1, RIPEMD

## MAC-Funktionen

- HMAC (128 bit)
- CMAC (128 bit)
- GMAC (128 bit)
- KMAC (256 bit)

## Signatur-Methoden

- RSA (3000 bit)
- DSA (3000 bit)
- Merkle-Signaturen
- ECDSA (250 bit)

## SHA-3 (Keccak)

- neuer Standard seit 2012
- Sponge-Funktion mit mehreren Runden in zwei Phasen:
  - 1.) Absorbing
    - Der State wird in zwei Teile geteilt:
      - Capacity  $c \rightarrow$  größeres  $c \Rightarrow$  sicheren
      - und Bit-Rate  $r \rightarrow$  größeres  $r \Rightarrow$  weniger Blöcke, schneller, bessere Performance
    - Input-Blöcke werden aufgefüllt  $\rightarrow$  Padding
    - $r$ -bit-Teil wird mit State ge-XOR-t
    - $c$ -bit-Teil bleibt unverändert
    - State wird mit  $f$  durchgemischt (Keccak Permutation)
  - 2.) Squeezing
    - Hashwert aus  $r$ -bit-Teil extrahieren
    - solange den State permutieren und den Output danach weiter extrahieren, bis die Länge erreicht ist

## Password-Hashing

- Probleme:
  - Hashfunktionen müssen schnell sein  $\rightarrow$  leichter für Brute Force Attacks
  - gleiches Passwort  $\Rightarrow$  gleichen Hash-Wert  $\rightarrow$  Dictionary Attacks
- Lösungen:
  - Kosten-Parameter für Zeit- und Speicherverbrauch-Tuning
  - Passwort mit Salt kombinieren  $\rightarrow$  unterschiedliche Hash-Werte trotz gleichem Passwort
  - zufällige Daten als zusätzlicher Input in der Hash-Funktion.
  - Für jedes Passwort neues Salt generieren!! Aber irgendwo speichern
- Sichere Passwort-Hashing-Algorithmen
  - bcrypt
  - Argon2
  - Scrypt
  - PBKDF2

## Message Authentication Codes (MAC)

- = Hash-Funktion, die zusätzlich einen secret key verwendet
- nur die beiden Kommunikationspartner kennen den secret key
- Symmetrisches Verfahren
- damit können unauthorisierte Modifikationen erkannt werden  $\rightarrow$  Integrität
- ermöglicht die Überprüfung, ob Daten authentischen Ursprungs sind  $\rightarrow$  Authentizität
- aber: keine Nicht-Abstreitbarkeit!
- keine Möglichkeit, durch eine Third-Party-Authority nachzuprüfen
- keine Zeitlichkeit
- viele moderne Chiffren kombinieren Verschlüsselung mit MAC
  - CTR (counter mode) mit CBC-MAC
  - GCM
  - ChaCha20 mit Poly1305
- keine simple Concatination:  $\text{hash}(\text{Secret} \parallel \text{Data})$ 
  - $\rightarrow$  Wenn der Angreifer die Länge von Secret  $\parallel$  Daten weiß, kann er nämlich  $\text{hash}(\text{Secret} \parallel \text{Daten} \parallel \text{mehr Daten})$  berechnen, ohne das Secret zu kennen
  - $\Rightarrow$  Length Extension Attack (SHA-1 und SHA-2 sind anfällig!!)

## Data Authentication

- Prozeduren, die garantieren, dass übertragene oder gespeicherte Daten nicht von autorisierten Personen manipuliert wurden
- basiert auf kryptografischen Schlüsseln, mit denen Checksums berechnet werden
- das geht symmetrisch und asymmetrisch
- dient zwei Security-Goals:
  - Integrität
  - Nicht-Abstreitbarkeit ( $\rightarrow$  NUR mit digitalen Signaturen)

### Verwendung in / als

- Angriffserkennung in Dateisystemen
- Absicherung von Software-Downloads (Updates, Patches)
- Absicherung von Kommunikationsprotokollen (z.B. TLS)

## Hashed Message Authentication Code (HMAC)

- Hash kann nur mit einem geheimen symmetrischen Schlüssel berechnet werden
- 2-mal XOR, 2-mal Concatination, 2-mal Hashing

## Poly1305-MAC

- verwendet 32-Byte-Secret-Key
- generiert 16-Byte-Authenticator
- schneller als HMAC

## Cipher Block Chaining MAC (CBC-MAC)

- berechnet MAC mit einer Block-Chiffre in CBC-Mode
- nur sicher bei Nachrichten mit fixer/bekannter Länge, sonst anfällig für Length Extension Attack!
- ① IV ist 0 am Anfang, wird mit Plaintext-Block 1 ge-XOR-t
- ② Ergebnis wird mit Key verschlüsselt
- ③ Ergebnis wird mit nächstem Plaintext-Block ge-XOR-t
- ④ MAC = Ergebnis nach letztem Block



# Digitale Signaturen

- = verschlüsselter Hashwert, den man ans Dokument dranhängt
- asymmetrisches Verfahren
- Enthält:
  - Timestamp
  - Person oder Ort → Servername
  - signierter Digest des Dokuments
- Weist nach:
  - Wann
  - Wer
  - Was

- Standards: PKCS #7

- WYSIWYS what you see is what you sign
  - man muss alles, was man signieren möchte, sehen können
    - es gibt aber Dokumente mit unsichtbaren Formaten z.B. Macro in Word, Javascript in Webseiten
  - verborgene Inhalte müssen entweder
    - angezeigt
    - eliminiert
    - oder auf ein Format ohne verborgene Inhalte gebracht werden
  - Erst signieren, dann verschlüsseln weil man sonst etwas signiert, das man nicht lesen kann

## Ablauf

- ① Alice generiert digitale Signatur
  - Dokument → **HASH** → Digest → **ENCRYPT** (Private Key) → Digital Signatur
- ② Alice sendet digitale Signatur + Dokument
- ③ Bob hashet das Dokument selber nochmal mit gleichem Algorithmus
  - Plaintext → **HASH** → Digest
  - Digital Signatur → **DECRYPT** (Public Key) → Digest
  - Wenn diese beiden gleich sind, wurde nichts verändert (keine Tampering)
- ④ Bob entschlüsselt die Signatur und vergleicht die Hashwerte

## Merkle Signature Scheme

- basiert auf Hash-Bäumen (Merkle Trees)
- Verwendet für Authentikation in Block Chains → z.B. Bitcoin
- Sicher vor Manipulation, da man kein Blatt ändern kann, ohne den ganzen Baum zu ändern
- ⊕ resistent gegen Quantum Computing
- ⊕ Effizienter als Hash-Listen
- ⊖ Anzahl möglicher Signaturen pro Public Key beschränkt auf die Anzahl der Blätter (2er-Potenzen)
  - wenn alle Blätter aufgebraucht sind, muss neuer Baum her

- ① Suche Key-Paare  $X_i$  (private),  $Y_i$  (public)
- ② Wähle  $i$  z.B.  $i=2$
- ③ Signiere das Dokument mit  $X_2 \rightarrow M$
- ④ Berechne den Public Key  $A_3$  von unten nach oben

$$\begin{aligned}
 A_0 &= \text{Hash}(Y_2) & \text{auth}_0 &= \text{Hash}(Y_3) \\
 A_1 &= \text{Hash}(A_0 \parallel \text{auth}_0) & \text{auth}_1 &= \text{Hash}(a_{0,0} \parallel a_{0,1}) \\
 A_2 &= \text{Hash}(A_1 \parallel \text{auth}_1) & &= \text{Hash}(\text{Hash}(Y_0) \parallel \text{Hash}(Y_1)) \\
 A_3 &= \text{Hash}(A_2 \parallel \text{auth}_2) & \text{auth}_2 &= \text{Hash}(\text{Hash}(\text{Hash}(Y_4) \parallel \text{Hash}(Y_5)) \parallel \text{Hash}(\text{Hash}(Y_6) \parallel \text{Hash}(Y_7)))
 \end{aligned}$$

- ⑤ Berechne Signatur  $\text{Sig} = (M, \text{auth}_0, \text{auth}_1, \text{auth}_2)$
- ⑥ Verifiziere  $M$  mit  $Y_2$
- ⑦ Prüfe/Verifiziere, ob  $A_3 \stackrel{?!}{=} (Y_2, \text{auth}_0, \text{auth}_1, \text{auth}_2)$

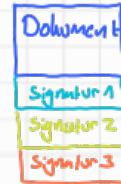
## XML-Signaturen

- ganze XML-Dokumente
- Teile von XML-Dokumenten
- andere Dateien
- Canonicalization nötig
  - bringt XML-Dokumente auf eine standardisierte Repräsentation
  - Problemursachen: Closing-Tags in XML können unterschiedlich dargestellt werden, obwohl sie semantisch gleich sind. Auch: whitespace characters
  - würde zu unterschiedlichen Hashwerten führen



## Parallel Signatures

- jeder signiert das Dokument unabhängig voneinander
- am Ende werden die Signaturen einfach alle ans Dokument angehängt
- Vorteil?
  - evtl. schneller?



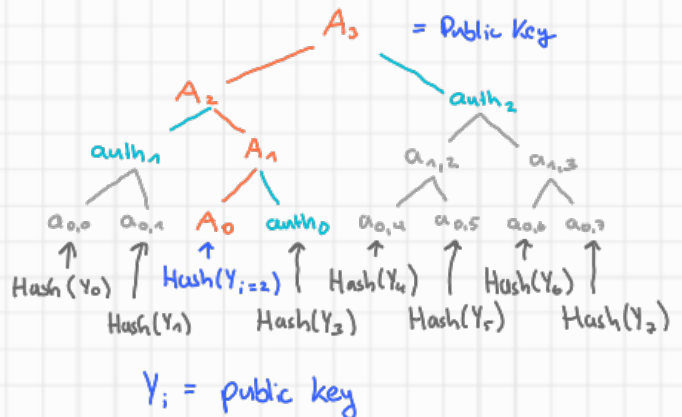
## Sequentielle Signaturen

- die Signaturen erfolgen nacheinander
- jeder signiert die vorherige Signatur mit
- Vorteil?
  - Man weiß, dass es den Vorgänger signiert hat
  - evtl. sicherer?



## Signaturerneuerung

- Warum?
  - veraltete / unsichere Operationen z.B. Hash, Verschlüsselung, Keylength
  - Änderungen im Datei- oder Signaturformat
  - neue Gesetze, die vorschreiben, dass Signaturen für eine längere Zeit verifiziert werden können sollen
- Sollte in sicherer Umgebung durchgeführt werden
- Sollte zertifiziert werden aus legalen Gründen
- ① Alte Signatur verifizieren
- ② Neue Signatur erstellen → enthält Dokument + alte Signatur
- Formatänderungen evtl. nötig

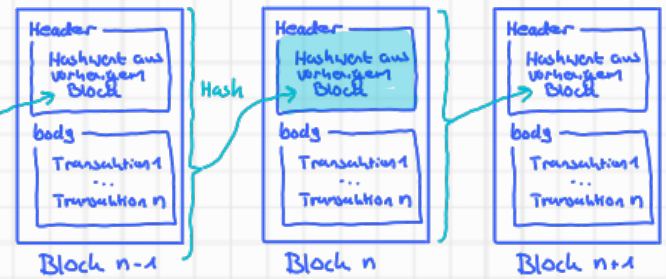


## Pseudonymization

= wenn Signaturen einen Public Key verwenden, der nicht einer konkreten Person zugeordnet ist

## Bloch Chains

- Integrity → Hashwerte
- Availability → Dezentralisation (dupliziert an mehreren Standorten)
- Confidentiality → schwierig zu implementieren, oft gar nicht erwünscht (Transparenz)  
 aber z.B. Vertrauliche Daten extern speichern, Trusted Execution Environments (TEE) sowie Multi-Party Computation (sMPC) → homomorphe Verschlüsselung  
 d.h. auf verschlüsselten Daten arbeiten, ohne sie zu entschlüsseln
- Authenticity → Private Signature Keys  
 • private Blockchain sollten Accounts mit Identifikation nutzen  
 • public Blockchain sollten Pseudonymität nutzen



## Public Key Infrastructure (PKI)

- Key- und Certificate-Management
- Interface für Trust Services → Generation, Verification, Revocation
- Bestandteile:

- Certification Authority (CA)  
 - erstellt Zertifikate
- Registration Authority (RA)  
 - Sprachrohr zwischen CA und Klienten  
 - Identifiziert Klienten
- Directory Service / Repository  
 - enthält Liste mit allen vergebenen Zertifikaten  
 - stellt Revocation List bereit, um Zertifikate zu sperren / zurückziehen (wie Bankkonto)
- Client Unit  
 - enthält die Applikation  
 - z.B. PC, Smartphone

- Personal Security Environment (PSE)  
 - Umgebung, in der Keys gespeichert werden  
 - z.B. ChipCard, Festplatte
- Timestamp Service (TSS)
- Revocation Instance (REV)
- Recovery Instance

Online Certificate Status Protocol (OCSP)

es gibt Onlinetools, mit denen man Zertifikate überprüfen/verifizieren kann

## Key- und Certificate-Generierung

- ① User generiert Schlüsselpaar und speichert es im KeyStore
- ② User stellt an die CA einen Certificate Signing Request (CSR), der enthält  
 • die Attribute des Zertifikats  
 • Public Key
- ③ User lässt sich von der RA identifizieren
- ④ RA bestätigt CA die Identität und den Public Key des Users
- ⑤ CA sendet Zertifikat an User, welches den Public Key enthält

## Zertifikate

- = digital IDs
- haben zeitlich begrenzte Gültigkeit
- können entzogen / widerrufen werden
- werden in Protokollen wie SSL, S/MIME, IPsec verwendet
- Standardformat: X.509 v3  
 ↳ Description Language: Abstract Syntax Notation (ASN.1)

## Zertifizierungsmodelle

- Web of Trust  
 ① Simple, flexibel  
 ② viele potenzielle Trust chains  
 ③ Keinen oder schwer zu bekommenen Beweiswert (evidential value)  
 ④ vertrauenswürdiges Pfad finden ist komplizierter

## Hierarchische Zertifizierung

- ① klare Strukturen und klare Verantwortungen
- ② Beweiswert (evidential value) in Zweifelsfällen
- ③ Overhead durch organisatorische Struktur



## Certificate Revocation List (CRL)

- Verlorener oder gestohlener (kopierter) Key muss gesperrt werden und andere User müssen gewarnt werden
- Enthält Liste aller Seriennummern von widerrufenen Zertifikaten
- IETF-Standard

## Im Directory Service einer CA gespeichert

- signiert (mit Zeitstempel) von der CA
- regelmäßig gesendet
- kann allerdings sehr lang werden
- Wie können Clients darauf zugreifen?  
 • Download  
 • Online-Status-Check durchführen (OCSP: Online Certificate Status Protocol) über den Link im Zertifikat

## European eSignature Directive

### → Electronic Identification Authentication and Trust Services (eIDAS)

- EU-Regelung für elektronische Identifikation, elektronische Signaturen und Trust Services für elektronische Transaktionen im internen Markt
- 3 Levels von Signaturen
  - Simple electronic signatures  
 ↳ Daten in elektronischer Form aus Dokument anhängen
  - Advanced electronic signatures (AdES)  
 ↳ Eindeutig auf einen identifizierbaren Unterzeichner zurückzuführen  
 ↳ so mit dem Dokument verbunden, dass alle Änderungen der Daten erkannt werden
  - Qualified electronic signatures (QES)  
 ↳ von einem qualified signature creation device erzeugt  
 ↳ basiert auf qualifizierten Zertifikaten für elektronische Signaturen  
 ↳ äquivalent zu einer handschriftlichen Unterschrift

## Bestandteile von eIDAS

- Elektronische Identifikation  
 ↳ EU-Anwohner haben eID-Funktionen
- Trust Services (= Anbieter qualifizierter Services)  
 ↳ unterstützen Erstellung und Verifikation von elektronischen Unterschriften (natürliche Personen) und von elektronischen Siegeln (juristische Personen) und von elektronischen Zeitstempeln  
 ↳ vergeben Zertifikate für Webare-Authentifikation  
 ↳ müssen selber auch den Zertifizierungsprozess durchlaufen

- ermöglicht Erstellung elektronischer Dokumente mit
  - elektronischen Unterschriften → Proof
  - elektronischen Siegeln → Authentifikation
  - Zeitstempeln → Proof of Creation
  - elektronische Lieferdienste mit Kassenzettel-Bestätigung
- z.B. Personalausweis, Elektronische Steuererklärungen (EStEn), ...



# Authentication

VS

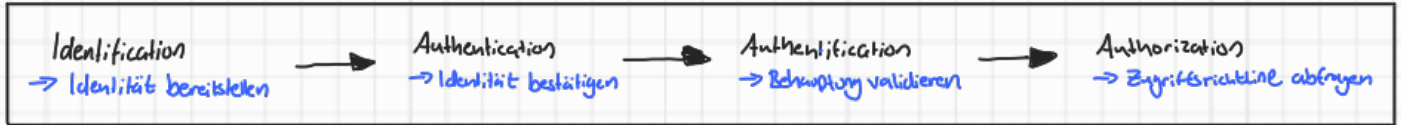
# Authorization

= Prozess, in dem die Identität eines Users, Prozesses oder Geräts verifiziert wird.

- ermöglicht durch: **Identität**

= Prozess, in dem überprüft wird, ob eine Person, IT-Komponente oder Applikation dazu berechtigt ist, eine spezielle Aktion durchzuführen oder auf bestimmte Ressourcen zuzugreifen

- ermöglicht durch: **Richtlinien**



- Möglich über...

- Wissen (z.B. Passwort)
- Besitz (z.B. Chipkarte)
- Persönliche Eigenschaften (z.B. Fingerabdruck)

Kombinationen möglich

## Password Authentication

- weit verbreitet, einfach, mobil

- Probleme:

- Password Cracking
- Wenn jmd. dein Passwort hat, hat er uneingeschränkten Zugriff → es ist schwer zu prüfen, ob jmd. mein Passwort geknackt hat!
- Passwort vergessen → sichere Recoveryprozeduren nötig
- Unverschlüsselte Übertragung übers Netzwerk

- Maßnahmen:

- Mindestlänge, Sonderzeichen, regelmäßige Änderung
- Blocken / Cool-Down nach mehreren gescheiterten Anmeldeversuchen
- Letztes Login-Datum anzeigen
- Speicherung als Hashwert → Salt verwenden

- Andere Maßnahmen:

- Einmalpasswörter, Transaction Numbers (TAN) → Online Banking
- Nonce, Captchas (Zusätzliche Aufgaben), Sicherheitsfragen, Zeitstempel

## Biometrische Authentifikation

- z.B. Fingerabdruck, Handvenen, Augen / Iris, Gesichtserkennung, Stimme, Unterschriftenerkennung, Verhalten beim tippen

- Probleme:

- kann gefälscht / abgefangen werden
- Implizieren von Daten, die den biometrischen Sensor umgeben
- meist schwer austauschbar
- unzureichende Erkennung → flächendeckende Überwachung möglich
- Referenzdatenbank stellt Sicherheitsproblem dar
- täuscht 100%ige Sicherheit vor

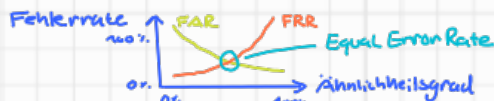
• Fehler:

### False Acceptance Rate (FAR)

fälschlicherweise Unautorisierte durchlassen → false positive

### False Rejection Rate (FRR)

fälschlicherweise Autorisierte verweigern → false negative



## Challenge - Response - Methode

- Ein Teilnehmer fordert den anderen auf, eine bestimmte Aufgabe zu erledigen, um zu beweisen, dass er eine bestimmte Information besitzt, ohne diese Information direkt mitzuteilen / zu übertragen

- geht mit symmetrischen und asymmetrischen Methoden

- geht ohne menschliche Interaktion

- z.B. Shared Private Key überprüfen:

- ① Sender übermittelt zufällige Nachricht an Empfänger
- ② Empfänger verschlüsselt Nachricht mit private Key und sendet sie zurück
- ③ Sender vergleicht Nachricht mit seiner eigenen verschlüsselten Nachricht → gleich: erfolgreich authentifiziert, ungleich: Misserfolg

→ z.B. Smart Cards und Computer am Arbeitsplatz

## Discretionary Access Control (DAC)

- benutzerdefinierte Zugriffskontrollen
- jeder Eigentümer kann die Rechte zu seinen Objekten an andere User übertragen
- dezentrale Vergabe der Zugriffsrechte → skalierbaren, leichter handhabbar
- beschränkt Zugriff auf Objekte basierend auf der Identität des Users → User-centric
- z.B. Lese-/Schreibrechte auf Unix-Systemen

## Mandatory Access Control (MAC) ♥ RuBAC

- Zentrale Zugriffskontrolle durch das System (z.B. ein Admin)
- basiert auf Regeln (rule-based) → mehr zentralisierten Aufwand
- Systemdefinierte Rechte überschreiben (dominieren) benutzerdefinierte Rechte
- globale Regeln und Security Classes
- Zugriffsbeschränkungen basierend auf Sensibilität der Daten
- Betriebssysteme oder Anwendungen müssen spezielle Maßnahmen und Services bereitstellen, um MAC durchzusetzen

## Umsetzung als Access Matrix

- Zwei Dimensionen: Subjekte (Wer) und Objekte (Was)
- Drinnen stehen dann die Zugriffsrechte (z.B. lesen, schreiben, ...)
- meistens „dünn besiedelt“

## Access Control List (ACL)

- Objektbasierte Ansicht: eine Liste pro Objekt
- Vorteil: Leichte Verwaltung und Widerrufung von Rechten
- Nachteil: Event. ineffizient, wenn es viele Subjekte gibt

File1 = { (user1, rw), (user2, r), (user3, rwd), ... }

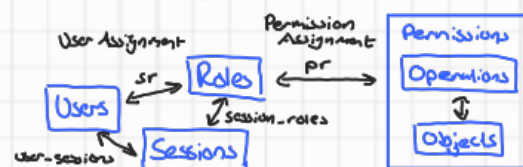
User1 = { (File1, rw), (File2, rwd), ... }

## Capabilities (Permissions)

- Manipulationssichere Access-Tickets, die einem Subjekt den Zugriff auf ein Objekt erlauben
- Vorteil: Flexibel, dezentral, geeignet für Delegierung
- Nachteil: Entzug von Rechten ist zeitaufwendig

## Role Based Access Control (RBAC)

- pr = permission to role, sr = subject to role
- dynamisch → häufigen Wechsel von Rollen (sr)
- User können mehr als eine Rolle haben (suchen sich über pro Session eine aus)
- Rollenmitgliedschaften können sich gegenseitig ausschließen → Static Separation of Duties
- Rollen haben eine Hierarchie → wenn  $R_i, R_j \in \text{Role}$  und  $R_i \neq R_j$ , dann hat  $R_i$  alle Rechte von  $R_j$  und mehr





## Zweifaktor-Authentication (2FA)

- Computer und Passwort-Knack-Algorithmen sind heutzutage zu schnell → einfaches Passwort evtl. zu schwach
- Zweiter Faktor ist oft ein **One-Time-Token (OTT)**, der über einen zweiten Kanal übertragen wird  
→ **Two Channel Authentication**
- Zweiter Faktor muss **unique** und nur eine **kurze Zeit lang** gültig sein.
- z.B. RSA Secure ID, Yubikey, TAN, SMS, Spezielle Handys

## Single Sign On (SSO)

- User kann auf alle Comput./Services (zu denen er autorisiert ist) zugreifen, ohne sich noch einer einmaligen Authentification jedes mal neu anmelden zu müssen
- User bekommt nach erster Authentification einen digitalen Token, der als digitale ID für die Anwendungen dient
- **Vorteile:**
  - + User muss sich nur einmal anmelden
  - + Passwort muss nur einmal übertragen werden
  - + nur ein Passwort nötig
  - + Zugriffsschutz / Blockierung eines Users an einer zentralen Stelle möglich
- **Nachteile:**
  - SSO-Server ist die Schwachstelle des Systems
  - keine Garantie der Availability: Setzt das SSO-System aus, sind alle Systeme blockiert!
  - Wenn jmd. eine SSO-ID stiehlt, hat er Zugriff auf viele Systeme!
- Beispiele: Google, Shibboleth, Windows Azure Active Directory, Kerberos, OpenID, Security Assertion Markup Language (SAML)

Federated SSO → Geht über Unternehmensgrenzen hinaus

## Identity Brokering

- Vertrauen zwischen versch. Providern
- ermöglicht Zugriff von extern authentifizierte Usern
- Providern können aber nicht direkt auf die Authentifications-Information der anderen Providern zugreifen

Single Sign Out → User loggt sich bei allen Services auf einmal aus

## Kerberos Authentication System

- Windows verwendet das als SSO-Protokoll
- Authentication Service von vertrauenswürdigen **Authentication Servern (AS)** gestellt } **Authentication der Subjekte**
- Access-Tickets für einen Service werden von einem **Ticket Granting Server (TGS)** gestellt } **Austausch von Session Keys**
- **Problem:** AS und TGS müssen 24/7 online verfügbar sein
- **Gut:** Es werden keine User-Passwörter übertragen  
→ verwendet nur Symmetrische Methoden

1. Klient kontaktiert mit dem AS-Key Nachricht mit Kerberos-Server (AS) aus und bekommt den TGS-Key einmündig beim Login
2. Klient kommuniziert mit TGS und bekommt bei Bedarf Tickets, also Session Keys für den jeweiligen Service
3. Kommunikation mit Service über Service-Key



**Sessions** = wenn ein User aktiv in einer Rolle ist

- der User soll pro Session nur eine Rolle haben
- der User hat dann nur die Rechte dieser Rolle
- möchte der User Rolle wechseln, braucht er eine neue Session  
→ also: neu anmelden
- der User kann nur in einer Rolle sein, der er angehört

## Vorteile von RBAC:

- flexibel, skalierbar, aufgabenorientiert und administrierbar
- Organisatorische Strukturen können direkt abgebildet werden  
→ gute Basis für ID-Management
- Rollen sind intuitiv an Workflows orientiert  
→ Rechte können nach dem **need-to-know**-Prinzip vergeben werden (d.h. ich bekomme nur die Rechte, die ich für den Task brauche)
- Änderungen der pr sind selten, aber Änderungen der Rollenmitgliedschaften sind passieren häufig
- Einfache und effiziente Verwaltung  
→ Widerrufung der Rechte automatisch bei Austritt aus Rolle

- **Gefahr:** Rollen wie Permissions verwenden  
→ kann zu einer viel zu großen Anzahl an Rollen führen!

## Rule Based Access Control (RuBAC) ♥ MAC

- Zugriff basierend auf Regeln
- Regeln beschreiben Situationen, in denen ein Subjekt auf ein Objekt zugreifen darf
- Regeln können als Richtlinien dargestellt werden → **Policy Based Access Control (PBAC)**
- Kann man gut mit User Rights umsetzen → mit MAC
- kann **schnell sehr komplex** werden  
→ v.a. wenn man viele neue Regeln hinzufügt, ohne alte zu löschen
- verwendet man in Firewalls, Routing, Cloud-Umgebungen
- ermöglicht durch **Open Policy Agent (OPA)**
  - z.B. in OPA Language Rego
  - evaluiert Query-Input anhand von Policies und Daten und entscheidet dann

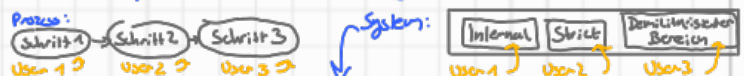


## Attribute Based Access Control (ABAC)

- Zugriff basierend auf Attributen und Policies (?)
- geschrieben in **extensible Access Control Markup Language (XACML)** oder **SAML**
- verwendet in z.B. API-Gateways von Microservices, Big Data Systems
- ① Subjekt fragt Zugriff an → z.B. Subjekt.name == Objekt.owner
- ② Access Control Mechanismus prüft Regeln und Environment Conditions und vergleicht Attribute von Subjekt und Objekt
- ③ Anhand dessen wird entschieden, ob autorisiert / Zugriff möglich

## Access Control Patterns

- **Least Privilege:** → Subjekt sollte nur die Rechte bekommen, die es für eine Aufgabe braucht  
→ höhere Stabilität und Sicherheit (z.B. nur als Admin einloggen, wenn man Admin-Zugriff braucht)
- **Need to Know:** → Zugriff wird nur gestattet, wenn Subjekt das zugeordnet braucht
- **Separation of Duty:** → Um bestimmte Aufgaben zu erledigen, wird mehr als ein User benötigt  
→ Schutz vor Betrug, Fehlern und Insider-Angriffen



- **Separation of Concerns:** → Computerprogramm in abgetrennte Bereiche teilen  
→ macht es schwächer, alles (auf einmal) zu hacken
- **Open Policy:** → alles, was nicht explizit verboten ist, ist erlaubt → bequemer  
→ Black Listing
- **Closed Policy:** → nur, was explizit erlaubt ist, ist erlaubt → sicherer  
→ White Listing
- **Dual Control:** → 4-Augen-Prinzip: Zwei oder mehr Personen werden benötigt, um sensible Daten / Informationen abzufragen

## Security Assertion Markup Language (SAML)

- SAML 2.0 ist ein XML-Framework für den Austausch von Informationen über Authentication UND Authorization
- OASIS Standard
- Angelehnt durch: SSO, Federated Identity, WebServices (SOAS)
- Identity Provider → Authentication (verifiziert Enduser) → beschreibt Profil eines Users
- Service Provider → Authorization (benötigt Verifikation vom Identity Provider) → beschreibt Permissions eines Users

## Open Authorization (OAuth)

- OAuth 2.0 ist Standard für Autorisierung von API-Zugriffen im Netz
- Ermöglicht die Delegation von Autorisierung
- Access Token enthält Autorisierung des Subjects + Permissions
- Refresh Token kann man benutzen, um neue Access Tokens zu bekommen
- Geeignet für serverseitige Anwendungen, z.B. Cloud native Services, wo Access Tokens sicher gespeichert werden können
  - Eigentümer kann einmaligen Zugriff auf Service erlauben
  - Server speichert Access- und Refresh-Tokens

## OpenID Connect

- erweitert OAuth mit SSO-Funktionen
- dezentralisiert → es gibt viele OpenID-Provider, zwischen denen man wechseln kann
- Authentication-Information werden im ID-Token gespeichert → JSON Web Token (JWT)
  - Signatur → Integrität
  - Claims → z.B. Username, E-Mail
  - Metadaten
  - optional: Verschlüsselung

1. Subject schickt Owner Authorization Request
2. Owner schickt Subject Zugriffsbefürwortung
3. Subject leitet Zugriffsbefürwortung an Authorization Server
4. Authorization Server schickt Subject Access Token
5. Subject leitet Access Token and Server mit Ressource weiter
6. Server schickt Subject die geschützte Ressource

## Zusammenspiel aus OAuth, OpenID und JWT

1. OAuth-Request: Eigentümer gibt Subject den Authorization Code
  2. Subject holt sich mithilfe des Authentication Codes die Ressource
- ja 4.4

- Access Token = Userinfo + Erlaubnis, auf eine Ressource zuzugreifen
- Refresh Token = für Erneuerung abgelaufener Access Tokens
- ID-Token = JWT-Token mit Identitätsinformationen

## Application Security

- Häufige Probleme / Fehler:
  - Anwendungen werden geplant, entwickelt und getestet, ohne auf Security zu achten
    - viele Vulnerabilities treten wegen schlechter, nachlässiger und unsicherer Programmierung auf
  - Security oft ad-hocmäßig implementiert, ohne die wirklichen Threats zu berücksichtigen
  - Security erst im Nachhinein als Erweiterung implementieren

## Open Web Application Security Project (OWASP)

- Listet alle paar Jahre TopTen-Vulnerabilities / kritische Schwachstellen in Webanwendungen auf

### 1. Broken Access Control

- Access Control ermöglicht es Usern, unbefugt auf fremde Ressourcen zuzugreifen und diese evtl. sogar zu modifizieren oder zu löschen
- z.B. Manipulation von Metadaten (wie z.B. in JWTs), keine Einschränkung / Deaktivierung von POST, PUT, DELETE -Endpunkten bei APIs
- Maßnahmen:
  - Zugriffskontrolle prüfen, bevor Daten geliefert bzw. Funktion ausgeführt wird
  - Sichere Access Control Patterns verwenden (s.o. z.B. low privileges, need to know & Co.)
  - Verwendung von CORS minimieren
  - Rate-Limits für APIs definieren
  - Zugriffsfehler loggen und Admins informieren
  - Unit- und Integrations-tests für Access Control durchführen

### Horizontale Privilege-Escalation

- Angreifer verschafft sich Zugriff auf Ressourcen anderer User mit gleichartigen Rechten

### Vertikale Privilege-Escalation

- Angreifer verschafft sich Zugriff auf Ressourcen für die höhere Rechte benötigt werden, z.B. Admin

### Cross-Origin Access

- Microservices und REST-APIs müssen oft domänenübergreifende Zugriffe durchführen: Cross Origin Resource Sharing (CORS)
- Angreifer können ungeschütztes CORS nutzen, um unbefugt Daten abzurufen
- Maßnahme: Same Origin Policy (SOP)

### Direkte Objekt-Referenzen

- Zugriff über Modifikation der URL möglich (z.B. ID ändern)
- Maßnahmen:
  - Vorheriger Zugriffskontrolle durchführen
  - Indirekte Objekt-Referenzen (nicht in URL)
- Maßnahmen:
  - Indirekter Zugriff auf Datennamen
  - Whitelists für Validierung
  - Normalisierung

Path Traversal / Force Browsing → direkter Zugriff auf Webserver-Dateien aufgrund von fehlender Validierung



### ③ Injection

- Bei Applikationen, die Code während der Laufzeit interpretieren können
- User-Input fließt direkt in den dynamisch erzeugten oder bearbeiteten Code ein
- Maßnahmen:
  - Input bereinigen, Metacharaktere entfernen oder escapen
  - Blacklisting → ungewünschte Zeichen entfernen oder escapen
  - Whitelisting → nur erlaubte Zeichen zulassen
  - Frameworks mit Sicherheitsfunktionen → LdapOwenyBuilder in Spring-LDAP
  - Web Application Firewalls (WAF)

z.B. SQL, Shell, LDAP, XML

### SQL Injection

- dynamisch erzeugte Datenbank-Queries durch User-Input manipulieren
- Folgen:
  - Freigabe vertraulicher Daten
  - Böswillige Manipulation oder Löschung von Daten
  - Schaden am System
  - Risiko in Webanwendungen umgehen
- Maßnahmen:
  - Sichere APIs verwenden, die keinen direkten Zugriff auf den SQL-Interpreter haben, sondern nur parametrisierte Interfaces nutzen
  - Vorgefertigte, parametrisierte Queries nutzen
  - Minimale Rechte für SQL-User, keine System-Accounts für Datenbank-User
  - Web Application Firewall (WAF)
  - Whitelist für Inputvalidierung
  - Data Washing → SQL-Metazeichen neutralisieren  
z.B. bei Anführungszeichen ohne zuckers Pünchen oder gesamten Input in ein neues Paar Anführungszeichen setzen

### Shell Command Injection / OS Command Injection

- OS-Commands werden von Webanwendung mit dynamischen Parametern aufgerufen
- Angreifer kann beliebige Befehle auf dem Server aufrufen und alle Daten und Teil der Hosting-Infrastruktur zerstören
- Maßnahmen:
  - Versuchen, komplett auf OS-Commands zu verzichten
  - Externe Programme direkt aufrufen (system, exec)
  - Funktionalität selbst programmieren
  - Mit Metazeichen umgehen
  - User-Input bei Command-Parametern verhindern
  - Sichere APIs verwenden

### Cross-Site-Scripting (XSS) → war 2017 noch ein eigener Punkt in der OWASP TopTen, seit 2021 gilt das als Injection

- Webseite unterscheidet nicht zwischen Code und Content
- Angreifer kann HTML-Konstrukte injizieren, die dann über die Webanwendung an die Browser anderer User weitergegeben werden
- Angreifer kann böswilligen JavaScript-Code im Browser anderer User ausführen
- Angreifer kann Cookies anderer User stehlen und Authentication umgehen
- Metacharakter-Problem und ein Output-Problem (SQL- und OS-Injection sind Input-Probleme)
- Anfällig sind Webanwendungen, bei denen User den Inhalt bereitstellen  
z.B. E-Mail, Diskussionsforen, Kommentarbereiche, Soziale Netzwerke
- Kann mit Social Engineering kombiniert werden
- Arten:
  - Stored Server XSS → Anwendung speichert ungeprüften User-Input in Datenbank. Zielt auf Server ab
  - Stored Client XSS → Anwendung speichert ungeprüften User-Input in Datenbank. Zielt auf andere User ab
  - Reflected Server XSS → Ungeprüfter User-Input landet direkt wieder im HTML-Output. Zielt auf Server ab
  - Reflected Client XSS → Ungeprüfter User-Input landet direkt wieder im HTML-Output. Zielt auf andere User ab
- Maßnahmen für Server
  - Output-Encoding auf Server
  - HTML-Encoding von User-Input, bevor er in den HTML-Output kommt } Reines Encoding reicht oft nicht aus!
  - Security-Encoding Libraries z.B. von Microsoft oder OWASP
  - Frameworks wie Ruby on Rails, React JS
  - User-Input-Daten von aktiven Browserdaten trennen
- Maßnahme für Client
  - Sichere JavaScript APIs
  - Same Origin Policy (SOP)
  - Strikte Content Security Policy
  - Input-Validation + Output-Sanitization

### ⑧ Software and Data Integrity Failures

- Vulnerabilities, die aus blindem Vertrauen in
  - Software-Updates
  - Kritische Daten
  - CI/CD-Pipelines
  - Plugins
  - Librariesentstehen → unautorisierte Änderungen am Code, an Konfigurationen oder an anderen Daten
- Problem: Unzureichende Überprüfung der Integrität
  - unbekannte Datenherkunft
  - manipulierte Software-Updates
  - kompromittierte Entwicklungsprozesse
- Supply-Chain-Angriff
  - ① Angreifer schleust böswilligen Code in einen zentralen Software-Lieferanten ein
  - ② Viele verschiedene Kunden trauen dem Software-Lieferanten und laden sich Schadsoftware herunter
- Maßnahmen
  - digitale Signaturen
  - nur vertrauenswürdige Quellen wie offizielle Repositories nutzen
  - sichere CI/CD-Pipelines mit strengen Zugriffskontrollen und Sicherheitsmaßnahmen

### ⑥ Vulnerable and Outdated Components

- Software enthält oft externe Libraries und Frameworks
- Manche haben Security-Features  
z.B. prepared statements in JDBC / Hibernate  
OAuth in Spring Boot
- Was mache ich aber, wenn diese veraltet oder veraltet sind?
  - patchen / updaten (nicht auf ältere Version zurückgehen)
  - Alternativen suchen, wenn nicht zu unbeständig
  - prüfen, ob mich die Schwachstelle überhaupt betrifft oder ob ich das betroffene Feature gar nicht nutze  
→ dann aber: Source dokumentieren und kommunizieren!  
(damit keiner doch das betroffene Feature einbaut)

## ⑦ Identification und Authentication Failures

### Broken Authentication

- Wenn Funktionen der Authentication und des Session-Managements nicht richtig implementiert sind, können Angreifer Passwörter oder Session-Tokens stehlen und so vorübergehend oder dauerhaft die Identität eines anderen annehmen
- Session-Management muss also sicher sein

### Session-Hijacking

- Angreifer umgeht Authentication, indem er sich Zugriff auf eine fremde Session-ID verschafft
- Maßnahmen:
  - Session-ID geheimhalten !!!
  - Sicheren Kanal zur Übertragung wählen → Verschlüsseln! , HTTPS
  - Lebenszeit einer Session begrenzen → Timeout
  - Bei jedem Login eine neue zufällige Session-ID generieren → keine persistierenden Sessions!
  - Session-Binding → Session-ID an IP-Adresse oder Browser-Fingerprint binden
  - Inhalt von Authentication Cookies verschlüsseln
  - Cookie-Attribute Setzen, z.B. Secure (nur HTTPS verwenden), HttpOnly (JavaScript kann das nicht lesen), domain <sup>(nur eine Domain kann das lesen)</sup>, expires (dauert...)
  - Anti-CSRF-Token (Cross Site Request Forgery) → Synchronisieren Tokens, um Session an einen fixen Client zu binden
  - Log-Out-Option!

## ⑧ Security Logging und Monitoring Failures

- Es besteht immer ein gewisses Rest-Risiko → Logging nötig, um Angriffe zu erkennen
- Logging = wichtigste Informationen, Angriffe und Eindringlinge zu erkennen → Basis für forensische Analysen
- Wichtigste Fragen
  - Was wird geloggt?
  - Was wird NICHT geloggt?
  - Wie sammle ich die Logs?
  - Wie analysiere und visualisiere ich die Logs?
- Vorgefertigte Toolchains wie ELK Stack können helfen (Elastic Search für Queries in Logs, Logstash fürs Sammeln + Transportieren, Kibana für Visualis.)

### Auditing

- wichtige User-Aktivitäten aufzeichnen
  - „gutes Verhalten“
- Traceability und Nachvollziehbarkeit
  - beweisen, dass etwas passiert ist
  - z.B. bei Bezahlungen

### Logging

- Alle Security-relevante Ereignisse aufzeichnen
  - Client/Server-Fehler
  - erfolglose Login-Versuche
  - Access Violations (wenn jemand Zugriff verweigert wird)

### Monitoring

- Performance einer Anwendung messen
- Abnormales Verhalten erkennen
  - Administratoren warnen / benachrichtigen
- langsame Antwortzeiten
- Speicher wird knapp

### Regeln

- böswilliges Verhalten identifizieren
- normales Verhalten wissen
- auf allen Anwendungsschichten loggen und auditieren
- Logging mit Monitoring integrieren

- Zugriff auf Logdateien einschränken
- Integrität der Logdateien sicherstellen → Hacker lieben diesen Trick! <sup>(einfach Log löschen)</sup>
- keine privaten oder sensible Daten in Logs schreiben
- Logdateien regelmäßig überprüfen

Error handling → s.u.

## Andere Threats für Webanwendungen

### - Clickjacking

- Angreifer legt unsichtbare / transparente Klickfelder übereinander und lässt den User da drauf klicken, um unsichtbare Aktionen durchzuführen
- Maßnahme:
  - Frame Busting → verhindert diese Frames
  - Content Security Policy (CSP) → in HTTP-Response-Header definieren, welche dynamischen Ressourcen der Browser laden darf <sup>(JavaScript, CSS)</sup>

### - Replay-Attacks

- Angreifer oder User schicken gleiche Request (vorherhinein) immer wieder
- Maßnahme: Anti-Replay-Token, Anti-CSRF-Token, Synchronisieren Token

### - Remote Code Execution (RCE)

- Angreifer kann Code auf den Maschinen seiner Opfer ausführen, z.B. Command-Injection, PathTraversal, Dateien ausführen
- absoluter Worst Case !!! <sup>und seine Privilegien erhöhen</sup>
- Maßnahmen:
  - Tiefgehende Defense auf allen Ebenen: Input, Server, Client, ...
  - Komponenten und Libraries regelmäßig updaten

### - Denial of Service (DoS)

- Kann man nicht verhindern → Notfallplan, um geeignet zu reagieren und Schaden abzuwenden
- Arten: Crash der Anwendung, Crash des Betriebssystem, Überladung der CPU, Überladung des Speichers oder anderer Ressourcen
- Maßnahmen, um Schaden zu reduzieren:
  - guten Code schreiben ...duh
  - Tests durchführen, auch mit vielen Anfragen
  - keinen Daten aus dem Netz brauen
  - komplexe Operationen nur erlauben, wenn du weißt, ob am anderen Ende ein vertrauenswürdiger Client sitzt
  - Anfragen validieren, bevor man Error Messages schickt → kontrolliertes Error Handling
  - Bei Angriff: Verbindung zu allen Clients ohne Authentication trennen
  - Rate Limiting, Torpits (Verlangsamungen)



# Security Issues laut Microsoft

- Authenticating Users
- Protecting Sensitive Data
- Preventing Parameter Manipulation
- Preventing Session Hijacking und Cookie Raping Attacks

- Validating Inputs
- Handling Exceptions
- Authorizing Users
- Authenticating and Authorizing upstream entities

- Auditing and Logging activity and transactions
- Encrypting or hashing sensitive Data
- Providing secure Configuration

## Gegner guten Codes:

- Ignoranz
- Unordnung
- Deadlines / Zeitdruck

## Schutzmaßnahmen

### - Best Practices:

- doppeltem Code vermeiden
- Funktionalitäten isolieren
- Länge von Code-Blöcken begrenzen
- Funktionalität von Funktionen begrenzen
- Scope von Variablen beschränken (global ist gefährlich)
- Unit Tests

### - Input-Validierung

- sicherstellen, dass Daten
  - den erwarteten Typ
  - das erwartete Format
  - eine gültige Länge
  - in den richtigen Range
 liegen.
- Integrität und Korrektheit der Daten prüfen
  - erfüllen sie die angegebenen Kriterien und Constraints?
- Checksums z.B. MAC

- immer gleich machen, bevor nächste Aktion durchgeführt wird
- nicht nur clientseitig! Jede Anfrage könnte manipuliert sein
- gleichzeitig auch gleich Authentisierung prüfen
- Alle möglichen Inputquellen prüfen, sogar solche mit vordefinierten Werten können manipuliert werden!
  - Dropdown-Menüs
  - Buttons + Checkboxes + RadioButtons
  - versteckte Felder

- URL-Parameter
- HTTP-Header
- Cookies
- Textinputs

⇒ Filtern über Blacklists und Whitelists

### - Error Handling s. oben zu Logging

- Software kann langsam werden, aber weder crashen noch inkonsistente Ergebnisse liefern
- Vermeiden, dass User durch einen Fehler mehr Rechte bekommt!
- Nach einem Fehler muss das System in einen stabilen und sicheren Zustand gebracht werden

#### • Regeln:

- Falsches Input nicht automatisch selber korrigieren (z.B. Passwörter)
- Wenn man einen Sicherheitsfehler findet:
  - 1) Bug beheben und nach ähnlichen Fehlern in restlichem Code suchen (v.a. bei Copy-Paste-Code)
  - 2) Bei der Korrektur sich so nah wie möglich an Vulnerability orientieren
  - 3) Ursache beheben, keine Symptombehandlung
  - 4) Bug in Testreihen aufnehmen

- System-Logs sind oft nicht ausreichend → Zusätzliches Logging in der Anwendung nötig, um Problemursachen zu finden

↳ Aber: dem Client keine detaillierten Error Messages schicken!

- keine Datenbankfehler, Zugriffswegungen, Datennummern, Table Names, Stack Trace
- sonst bekommt der Angreifer Informationen über die interne Struktur
- alle Fehler abfangen und nur eine allgemeine Error Message Page anzeigen
- detaillierte Fehlerbeschreibung kommt in die Log-Dateien

### → Mitigable Logging Risks:

- Rollenbasierte Access Control auf Log-Dateien
- Integrität der Log-Dateien bewahren
- Backup der Log-Dateien

### - Sichere Administration und Operation / Bedienung

- Auch gute Software kann schlecht bedient werden

#### • Sichere Infrastruktur nötig

- Firewalls (schützt ganzes Netzwerk auf Ebene der Netzwerk- und Transportschicht)
- Web Application Firewall WAF (schützt konkrete Anwendung vor spezifischen Angriffen auf Anwendungsschicht)
- SSL Gateways und Access Gateways
- Security Zones
- Remote Access für Administration besonders gut schützen!!
- Incident Response and Emergency Management einrichten

#### • Change Management

- viele Vulnerabilities treten durch Zugfixes oder Hinzufügen neuer Features auf
- Alle Änderungen tracken und testen!! Erst nach dem Testen deployen

#### • System Hardening

- Restriktive Konfigurationen, um die Angriffsfläche aller deployten Systeme zu reduzieren
- unsichere Default-Einstellungen ändern
- Zugriffsrechte anpassen
- Unnötige Services deaktivieren
- Vulnerability-Scans durchführen

#### • Decommission

Wenn man das System stilllegt, sollte man:

- Alle Authentication und Authorizations-Einstellungen entfernen
- Konfigurationen zurücksetzen → Firewall, offene Ports, DNS-Einträge
- Daten löschen (oder abspeichern)

#### • Multi-Layer-Defense

- Wenn ein Sicherheitsmechanismus fehlschlägt, sollte sich ein anderer um das Problem kümmern
- z.B. wenn SQL-Input-Validierung fehlschlägt, die Administrationsrechte der Datenbank einschränken



# Security on Cloud Platforms

- Cloud-Plattformen ermöglichen Continuous Integration CI und Continuous Deployment CD, was zwar flexibel ist, aber mehr Sicherheitsrisiken birgt
- Die DevOps - Rolle im Software-Engineering ist wichtig aufgrund der Micro-Services-Architektur auf Cloud-Plattformen, bei der die Bereiche der Entwicklung (Development) und Bedienung (operation) immer mehr verschmelzen
- Auf Cloud-Plattformen sollte so viel wie möglich automatisiert werden
  - Verbesserte Traceability von dem, was passiert
  - reduziert Fehler durch menschlichen Eingriff

## - Secure operations:

- Klare Verantwortungen
- Patch-Management für Infrastruktur und Tools von Drittanbietern
- Change Management s.u.

## - Schutz der CI/CD-Pipeline

- Ziel: aus vertrauenswürdigen Quellen deployen
- Risiko: ungetesteter oder unsicherer Code
- Maßnahmen:
  - Versionkontrolle in GitHub
  - 4-Augen-Prinzip

• ZFA

• automatische Tests

• Security checks

• Artefakt-Repository

• Rollback-Optionen

• Image-Registry

## - Security von Images und Containern

- Risiko: unautorisierte Änderungen in Produktionsumgebung
- Maßnahmen:
  - Sichere Basis-Images / Basis-Container verwenden
  - Vulnerability-Checks für Libraries, externe Services, APIs, Frameworks, Konfigurationen

## - Netzwerk-Security

- Schutz von anderen Anwendungen, Clients, Angreifern auf der Plattform
- virtuelle Isolation
- Eingangs- und Ausgangskontrolle

## - Zugriffs- und Account-Management

- Rechte und Rollen auf verschiedenen Ebenen kontrollieren
- Unterscheidung zwischen Technischen Accounts und Funktionalen Accounts
  - Verwalten die Cloud-Infrastruktur, GitOps

## Funktionalen Accounts

→ Verwaltung der Anwendung

- User-Management
- Reporting
- Hotline
- Data-Services
- Billing

## - Secret-Management

- Verwalten und schützen Passwörter, API-Keys, Access-Tokens
- Maßnahmen: Schlüssel versiegeln, Key Vault, Secret Manager

## - Cluster Backup und Recovery

- System neu starten
- Fast Failover (wenn eine Instanz der Ressource ausfällt, schnell zu einer anderen im Cluster wechseln)
- Datenverlust verhindern
- In Cloud schwieriger als lokal

# IoT - Security

- viele Geräte, viel Kommunikation, viele Daten

⇒ viele neue Herausforderungen

- Geräte schützen, die keinen physischen Zugriffsschutz haben
- (Alte) Geräte updaten, um Bedrohung langfristig zu ermöglichen
- Autonome Systeme können unabhängig Entscheidungen treffen
- Viele Standards haben keine Sicherheitsfeatures
- Viele billige Produkte für Konsumenten

## • Threats:

- Geräte-Hijacking → Botnets
- Informationslecks
- Unterbrechen von Services / Funktionalität

## • Vulnerabilities:

- Geräte sind dem WWW ausgesetzt
- Unsichere Web-Interfaces mit schlechtem Session-Management oder Default-Credentials
- unsichere Frameworks oder Protokolle
- veraltete Geräte
- sonstige typische Schwachstellen wie fehlendes Logging, kein Access Control, unsicherer Speicher, hardcodierte Passwörter

- reines Software-Engineering reicht für IoT nicht aus, zusätzlich: Systems-Engineering nötig!

## • Maßnahmen:

- Komplette Kommunikation verschlüsseln
- Sicheres Key-Management
- gegenseitige Authentifikation
- keinen unautorisierten Code ausführen
- sichere und regelmäßige Updates
- Code signieren
- least privilege
- Erkennung von Eindringungsversuchen
- Geräte als Client (nicht Host oder Server!) implementieren

## OWASP Top 10 für IoT:

- ① Schwache, erratische oder hardcodierte Passwörter
- ② Unsichere Netzwerkservices
- ③ unsichere Interfaces
- ④ Fehlen von sicheren Update-Mechanismen
- ⑤ Verweigerung von unsicheren oder veralteten Komponenten
- ⑥ Unzureichender Schutz der Privatsphäre
- ⑦ Unsicherer Transfer und unsichere Lagerung von Daten
- ⑧ Fehlen von Geräte-Management
- ⑨ Unsichere Default-Einstellungen
- ⑩ Fehlen von physischer Hürde



# Secure Software Engineering

## - Secure Software Development Life Cycle (SSDLC)

- ① Sicherheitsanalyse → alle Security Requirements sammeln und analysieren
- ② Sicherheitsdesign → sichere Architektur und Software-Änderungen entwerfen
- ③ Sichere Entwicklung → Sichere Coding Practices
- ④ Sicherheitstesten → Automatisierte Security Tests
- ⑤ Deployment und Tracking → Potenzielle Security-Bugs erkennen und fixen

Sicherheit von Anfang an berücksichtigen!

## ① Sicherheitsanalyse

### - Protection Requirements

- Kritische Informations-Objekte identifizieren
  - anhand von CIA bewerten
  - Welchen Schaden könnte entstehen, wenn diese Ziele nicht erfüllt werden?
- Use-Cases bewerten
  - welche Use-Cases können großen Schaden anrichten, wenn die Requirements nicht erfüllt werden?
  - Auch technische Use-Cases betrachten: z.B. Key-/Zertifikatsmanagement, Systemadministration, Authorization Assignment
- Rollen und Rechte im System verteilen
  - Welche User und Rollen gibt es?
  - Wer darf was machen?
  - Access Control - Prinzipien (→ need to know, segregation of duties, ...)

### - Risiko-Analyse

- umfassende Risk/Threat-Analyse sind zeit- und kostenintensiv
- oft ist der Kunde / Client nicht dazu bereit → programmatische Umsetzung finden und auf die wichtigsten Risiken konzentrieren
- Risiko muss von den verantwortlichen Parteien (ISO, DPA, PO, Management) bewertet werden
- Transparenz über die Risikobewertung schaffen

### - Threat-Analyse

- Threats identifizieren und analysieren
  - Threat Modeling - Risikoanalyse
- Microsoft Threat Model: STRIDE
  - ⑤ Spoofing → Authenticity
    - Ein User kann nicht zu einem anderen User werden oder Attribute eines anderen Users annehmen
    - Session Hijacking / Identity Theft
  - ⑥ Tampering → Integrity
    - böswillige Modifizierung persistenter Daten und von Daten im Netzwerk
  - ⑦ Repudiation → Non-Repudiation
    - User können abstreiten, etwas getan zu haben bzw. eine Transaktion durchgeführt zu haben, wenn die Aktivität nicht ausreichend aufgezeichnet wird
    - man kann nicht behaupten, dass jmd. etwas getan hat
  - ⑧ Information Disclosure → Availability
    - Offenlegung von Informationen an Personen, die keinen Zugriff darauf haben sollten → z.B. Log-Files oder Error Messages
  - ⑨ Elevation of Privilege → Authorization
    - unprivilegiertes User verschafft sich privilegierten Zugang (z.B. Admin-Rechte) und kann damit das ganze System gefährden oder gar zerstören

### • Threat Modeling in 5 Schritten:

- ① Security Requirements definieren
- ② Anwendungsdiagramm erstellen, System in Komponenten und Safety Zones aufteilen
- ③ Threats identifizieren
- ④ Möglichen Schaden durch Threats reduzieren
- ⑤ Validieren, ob Threats eingedämmt wurden

### • Alternative Herangehensweisen

- Misuse-Cases
- Attack Trees
- Threat Catalogs

### - Security Requirements

- Was soll das System tun?
- Welche Ziele sollen erreicht werden?
- abstrakt
- Ergebnisorientiert, nicht Lösungsorientiert

### - Security Measures

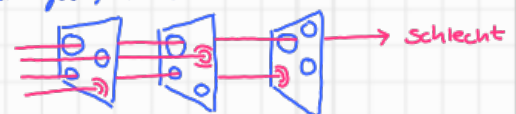
- wie setzt man die Requirements technisch oder organisatorisch um?
- spezifisch
- Lösungsorientiert

## ② Sicherheitsdesign

- Es gibt viele Prinzipien, nach denen man designen kann: z.B. OWASP, NISSE, CSA
  - ① Zero Trust: alle User, Geräte und Netzwerke gelten als ungläubwürdig, bis sie sich verifizieren
  - ② Keep it simple
  - ③ Secure Defaults: nicht jedes Feature sollte standardmäßig aktiviert sein
  - ④ Open Design: keine Security by Obscurity → Kennhoff
  - ⑤ Fail Secure: Fail-Saves → Anwendung immer in einem sicheren Zustand halten, um keine Daten zu verlieren
  - ⑥ Angriffsfläche minimieren
  - ⑦ Psychologische Akzeptanz: Userfreundlichkeit berücksichtigen, z.B. Iris-Scans sind weniger akzeptiert als Fingerabdruck-Scans

### • Defense-in-Depth: mehrere Sicherheitsschichten

- keine Maßnahme bietet absoluten Schutz → wenn möglich lieber viele verschiedene Maßnahmen mit verschiedenen Techniken
- Angenommen, ein Security-Level wurde angegriffen → Wie können wir trotzdem unser System schützen
- Schweizer-Käse-Modell



- Architektur in sichere und verlässliche Komponenten aufteilen
  - versichern, dass nur „clean“ Messages bis zu den Komponenten gelangen
  - Verlässliche Komponenten machen semantische und technische Checks
  - und verwenden sichere und verschlüsselte Protokolle

z.B. HTTPS mit Perfect Forward Secrecy, Authentication, Rate-Limiting, Torpits, One-Time-IDs, Signaturen

## ③ Secure Development



## ④ Sicherheitstests

- Andauernde automatische und manuelle Tests sind wichtig, um Fehler in Software rechtzeitig zu entdecken und zu beseitigen

- Viele verschiedene Arten von Tests:

- Unit Test
- Component Test
- Usability Test
- Regression Test (ob nach Bugfix der Rest noch passt)
- System Test
- Integration Test
- Acceptance Test
- Load Test
- Test-driven development
- Robustness Test

### - Automatische Tests

z.B. CI/CD-Pipeline Checks wie: Funktionalität → Werkbarkeit → Sicherheit → Compliance → Performance → ...

• Produktqualität bestimmt sich automatisch

#### • Static Application Security Testing (SAST)

- automatisierte statische Analyse-Tools
- gibt zur Compile-Zeit Warnungen, wo Bugs anfallen können
- z.B. SonarQube, FindBugs, Fortify

#### • Dynamic Application Security Testing (DAST)

- Web Security Scanner, Vulnerability Scanner: Computerprogramme, die Systeme auf bekannte Vulnerabilities untersuchen, z.B. OWASP ZAP, Nessus, Qualys

#### Passive Scans

- Scannen die Daten zwischen Browser und Webserver nach Mustern
- ändern weder Request noch Response
- passiert im Hintergrund
- weniger risky

Versucht z.B. Fehler zu provozieren

#### active Scans

- sucht typische Vulnerabilities mithilfe von üblichen Angriffen
- NIEMALS unerlaubt auf Webanwendung Dritter anwenden!

### - Individuelle Analysen

#### • Penetration Test

- Alle Systemkomponenten mit den gleichen Tools und Methoden testen, die ein Angreifer / Hacker verwenden würde
- Blackbox- oder Whitebox-Test (man kennt die Architektur nicht vs. schon)
- muss im Voraus geplant werden (Scoping, Vorbereitung, Vor-Analyse, Analyse, ...)
- muss von Professionals durchgeführt werden

#### • Fuzz Testing (Fault Injection)

- generiert (semi-)automatisiert ungültige / unerwartete / zufällige Inputdaten
- beobachtet Exceptions, Crashes oder fehlendes Error Handling → Monitoring

#### • Security Code Review

- gegenseitige Reviews sind zusätzlich zu SAST nötig

### - Security Testing Tools

#### • Attack Proxies für dynamische Analysen

→ Nessus, Qualys, Burp Suite, OWASP ZAP

#### • Software Composition Analysis (SCA)

- Tool, das in den Dependencies eines Projekts nach öffentlich bekannt gemachte Vulnerabilities sucht
- Ergebnis: Liste an Komponenten mit Vulnerabilities und einer Score, der sagt, wie kritisch die Schwachstelle ist mit Referenzen zu CVE/CWE
- z.B. OWASP Dependency Check

OWASP ZAP  
Attack Proxy

- kann aktive und passive Scans durchführen
- kann Fuzz machen
- nur mit expliziter Erlaubnis durchführen!
- nur auf Testsystemen durchführen!

Mitre.org

### Bekannte Vulnerabilities

- Es gibt öffentliche Datenbanken / Listen mit bekannten Vulnerabilities → können alle einsehen (auch Hacker)

#### - Common Weakness Enumeration (CWE)

- Sammlung generischer / produktunabhängiger Vulnerabilities
- Enthält:
  - Beispiele
  - Zusammenhänge mit ähnlichen / verwandten Vulnerabilities
  - Referenzen auf Literatur mit mehr Info
  - Maßnahmen, um Vulnerability zu erkennen
  - Maßnahmen, wie man Vulnerability vermeiden / verhindern kann

#### - Common Vulnerability Enumeration (CVE)

- Sammlung produktspezifischer Vulnerabilities für
  - Betriebssysteme
  - Sprachen
  - Programme
  - Frameworks
  - etc.
- Namensschema: CVE - <Jahr> - <Nummer>
- Nicht vollständig, aber allgemein die vollständigste Liste

#### - Common Vulnerability Scoring System (CVSS)

- Bewertungsschema um Schwere einer Vulnerability einzuschätzen → von 0.0 (unkritisch) bis 10.0 (extrem kritisch)
- Erster Referenzpunkt, aber man sollte sie selber nochmal im Projektkontext bewerten
- Enthält viele Parameter, z.B.
  - Attack Vector (Netzwerk, benutzbares Netzwerk, Lokal, Physisch)
  - Angriffskomplexität
  - benötigte Rechte
  - ob User-Interaktion erforderlich ist
  - Scope
  - wie stark welches CIA-Ziel betroffen ist
- Es gibt Online-Rechner, mit denen man den Score berechnen kann → CVSS Version 3

### • Hacker-Paragraph § 202c

- Auspähen von Daten (§ 202a) } Straf-
- Abfangen von Daten (§ 202b) } teilen
- Herstellen / Verschaffen / Verleihen / Verbreiten / Zugänglichmachen von Passwörtern, Sicherheitscodes oder Computerprogrammen, die sonst kaum - Bis zu 1 Jahr Freiheitsstrafe oder Geldstrafe
- Allein die Vorbereitung und der Besitz von Hacker-Tools ist bereits strafbar

### OWASP Dependency Check

• verschiedene Scan-Optionen, z.B.

- Command-Line-Tool
- Maven-Plugin
- Gradle-Plugin
- Archive wie .zip .war .ear
- .NET Assembly-Dateien
- JavaScript-Dateien

• False Positives und False Negatives berücksichtigen  
↳ kann man in Konfiguration einfach entfernen  
↳ v.a. aufpassen, wenn 0 Ergebnisse gefunden wurden

• Was tun, wenn ich Vulnerability gefunden habe?

#### ① Informationen sammeln

- welche Komponente ist betroffen?
  - was ist der Attack Vector? (=Methode, über die Exploit genutzt wird)
  - was impliziert das?
  - gibt es Updates / Workarounds?
  - gibt es wirklich Exploits?
- erstmal in CVE schauen

#### ② Bewertung

- Wie wirkt sich Vulnerability auf die Sicherheit des Systems aus?
- Was sind angemessene Maßnahmen?  
→ langfristige und kurzfristige
- Library updaten oder Alternative suchen

#### ③ Maßnahmen umsetzen



← wenn nicht von Anfang an verschlüsselt

# Secure Socket Layer (SSL) / Transport Layer Security (TLS)

Ursprünglicher Name von Netscape 1994 - 1999

Offizieller Name 1999 - heute

- Zwischen Transport - und Anwendungsschicht (③ und ④)
- Confidentiality → Hybrides Kryptosystem (asymmetrischen Schlüsselaustausch + symmetrische Verschlüsselung)
  - ⇒ geheimer Server-Key muss geschützt werden
  - ⇒ zusätzliche Rechenpower wird für die kryptografischen Operationen benötigt
- Integrity → Verwendet MACs als kryptografische Checksums
- Authenticity → Verwendet Zertifikate
  - ⇒ Server-Zertifikate müssen installiert und gemutet werden
- HTTPS Sendet HTTP-Request und -Responses über SSL/TLS (Port 443)
  - mit Browser-Plugin HTTPS Everywhere kann man TLS immer erzwingen
- Ermöglicht geschützten Datentransfer, bietet aber keine Ende-zu-Ende-Sicherheit (endet bei der Firewall)
- HSTS-Flag (HTTP Strict Transport Security) kann Browser dazu zwingen, nur verschlüsselte Verbindungen einzugehen

## SSL/TLS - Protokoll-Layers

### - Handshake-Layer

- Koordiniert und managt Informationen über die Zustände von Client und Server
- Authentifikation, Protokoll-Negotiation und Informations- und Schlüsselaustausch

### - Record-Layer

- teilt Datenpakete in Blöcke auf und komprimiert sie in SSL-Records
- berechnet die Checksum
- verschlüsselt Daten, inklusive MAC
- bekommt Key vom Handshake-Layer

- ① Fragmentation
  - ② Compression
  - ③ MAC
  - ④ Encrypt
  - ⑤ SSL Record Header anhängen
- ← TLS macht das falsch!  
Normal macht man Encrypt - then - MAC!

Heartbleed Exploit 2014 → kein Designfehler im Protokoll, sondern Implementierungsfehler

- OpenSSL (eine Implementierung vom TLS-Protokoll) hat ein Heartbeat-Feature eingebaut, das TLS-Verbindungen aufrecht erhalten lassen hat
- Die Heartbeat-Funktion besteht daraus, dass Client Payload mit beliebigem Inhalt geschickt hat und den Server hat das dann zurück geschickt
- Problem: Im Payload gibt es zwar ein Feld, das angegeben hat, wie lange der Payload ist - allerdings wurde nie überprüft, ob der Payload wirklich diese Länge hat
- Angriff: ① Angreifer schickt 1 Byte Payload, gibt aber 64 kB als Länge an  
② Server schreibt nur 1 Byte in den Speicher, schickt aber 64 kB aus dem Speicher zurück  
③ In den zurück geschickten Daten könnten z.B. Passwörter sein

## POODLE Attacke

Padding Oracle On Downgraded Legacy Encryption

- Vulnerability bis TLS 1.2 / SSL 3.0
- Man-in-the-Middle-Angriff
  - ① Browser dazu zwingen, Javascript-Code auszuführen
  - ② viele Requests durchführen / ausprobieren
  - ③ URL modifizieren, Blocks aus dem verschlüsselten Request in das Padding kopieren
  - ④ Server antwortet mit MAC oder Padding ist falsch
  - ⑤ Nach vielen modifizierten Requests mit Padding-Fehlern können Angreifer das Padding heraus und ein paar Bytes des Plaintexts aus dem vorherigen Block erraten

→ Problem: Sie haben zuerst MAC, dann verschlüsselt, obwohl man zuerst verschlüsselt und dann absichert!  
→ Designfehler im TLS-Protokoll

## Perfect Forward Secrecy

- Client und Server einigen sich auf einen gemeinsamen Key, der niemals über Netzwerk geht und nach der Session zerstört wird
- nachträglich können keine Nachrichten entschlüsselt werden
- Diffie-Hellman ist perfect forward secret, da der Schlüssel nie über Netzwerk übertragen wird

## Verbesserungen in TLS 1.3

- viele unsichere Krypto-Algorithmen wurden abgeschafft (z.B. SHA1, MD5, RC4, CBC)
- nur noch AEAD-Algorithmen für symmetrische Verschlüsselung erlaubt, da diese integrierte MACs haben (in der richtigen Encrypt-then-MAC-Reihenfolge)
- Verbesserter Handshake
- Verbesserte Performance, weniger Roundtrips
- Nur noch ECDH statt RSA als Schlüsselaustausch

### Elliptic Curve Diffie Hellman

- Client und Server berechnen gemeinsamen private Key selbstständig und nur public Parameter gehen über Netzwerk
- Schlüsselaustausch ist volatil, d.h. ist nur für eine Session lang gültig, weil jedes Mal ein neues Schlüsselpaar erzeugt wird

### RSA-Handshake

- Client erzeugt gemeinsamen private Key, verschlüsselt ihn mit public Key von Server und schickt ihn über Netzwerk
- Wenn ein Angreifer die Übertragung antzeichnet und später den private Key herausfindet, können nachträglich alles entschlüsseln

## Handshake-Problem

- Vor TLS 1.3 konnten noch unsichere Cipher Suites mit unsicheren kryptografischen Algorithmen verwendet werden
- Nur Server signiert das Protokoll → Man-in-the-Middle-Angriffe möglich
- FREAK-Angriff (Factoring RSA Export Keys) → Downgrade-Angriff
  - ① Man-in-the-Middle man manipuliert Cipher-Suite-Aushandlung und erzwingt Verwendung eines unsicheren Algorithmus z.B. RSA 40bit
  - ② Man-in-the-Middle kann schwachen Key mit 2<sup>o</sup> Möglichkeiten relativ leicht mit Brute Force herausfinden

## Grenzen von SSL/TLS

- ✓ Traffic-Flow-Analysen möglich
- X Firewall können Traffic nicht mehr analysieren
- X Ohne Client-Zertifikate keine Haftung für den übertragenen Payload
  - dafür bräuhete man eine gegenseitige TLS-Verbindung
- / Sind Zertifikate überprüft?
- / Werden Warnungen ignoriert?
- / Sind Master-Key und Session-Key sicher abgespeichert?

← hatte von Anfang an eine Verschlüsselung integriert (aber nun eine schlechte)

# WLAN / Wi-Fi

- Kabellose Kommunikation erschwert Bewahrung von Integrity und Confidentiality, da Man-in-the-Middle-Angriffe leichter sind
- Angriffsmöglichkeiten:
  - War Driving: Angreifer sucht aktiv nach offenen / ungeschützten Netzwerken, um sich da rein zu hacken
  - Rogue Access Point: Angreifer erstellt einen neuen, unsicheren Access Point und hofft, dass Unvorsichtige diesen ins Netzwerk lassen
  - Evil Twin: Angreifer erstellt neues, unsicheres Netzwerk, das so aussieht wie ein legitimes existierendes und wartet passiv darauf, dass sich Leute verbinden
  - Honeypot Access Point: Angreifer erstellt ein neues, offenes, unsicheres Netzwerk, lockt Unvorsichtige mit „Free-Wi-Fi“ und wartet passiv darauf, -“-

## • Wired Equivalent Privacy (WEP)

- Unsichere Verschlüsselung: RC4 (Stream Cipher) hat zu kurze Schlüssel, zu kurzen IV
- Nur eine recht simple Checksum
- Schlechtes Key-Management: Nur ein Key pro Netzwerk
- Authentication und Verschlüsselung verwenden den gleichen Key
- Access Point muss sich nicht authentifizieren

## • WiFi Protected Access 2 (WPA2)

- + Sicherere Verschlüsselung: AES (mit CCM-128) statt RC4 → Längere Schlüssel, längerer IV
- + Verwendet eine kryptografische Checksum → Message Integrity Check (MIC = Checksum)
- + Keys werden dynamisch erzeugt → Individuelle Session Keys mit shared Key
- + Master Key erneuert sich andauernd (mind. einmal pro Stunde)
- + Bessere Authentication mit drei Möglichkeiten:
  - WPA PSK → Pre Shared Key
  - WPA EAP-TLS → Basiert auf Zertifikaten und public Keys
  - WPA PEAP → Basiert auf Passwörtern
- + Paketnummern wird inkrementiert und alte Pakete werden ignoriert, um Replay-Angriffs zu verhindern
- + Verwendet Cyclical Redundancy Check (CRC) und MIC  
⇒ man kann zwischen Übertragungsfehler (MIC + CRC falsch) und Manipulation (nur MIC falsch) unterscheiden

### Master Key

= Symmetrischen Schlüssel, mit dem andere Keys generiert bzw. geschützt werden

## - Key-Reinstallation-Attacke KRACK - Angriff (2017)

- Angriff nur unter bestimmten Bedingungen möglich: Geräte müssen in unmittelbarer Nähe sein und dürfen nicht gepatcht sein
- Ermöglicht HTTP-Injection und Replay-Angriffs
- Probleme von WPA2 beim 4-Way-Handshake:
  - Verschlüsselung über XOR: Verwende ich den gleichen Schlüssel wiederholt (z.B. weil ich eine Nonce wiederhole), mit anderen Klartexten, kann ich den Schlüssel herausbekommen
  - Client akzeptiert und installiert basis verwendeten Schlüssel erneut, wenn man die entsprechende Handshake-Nachricht erneut sende
- Grober Ablauf:
  - ① Angreifer fängt Handshake ab
  - ② Angreifer schickt abgefangene Handshake-Nachricht erneut los, um alten Key / alte Nonce erneut zu installieren
  - ③ Client verwendet alten Key / alte Nonce für Verschlüsselung
  - ④ Angreifer kann Nachrichten entschlüsseln oder manipulieren und Replay-Angriffs durchführen

## • WiFi Protected Access 3 (WPA3) → seit 2019

- + Sicherere Verschlüsselung: Galois Counter Mode GCM-256 statt CCM-128 wie WPA2
- + Sichererer Handshake: Simultaneous Authentication of Equals (SAE) mit dem Dragonfly-Protokoll statt veralteten 4-Way-Handshake von WPA2
  - verhindert Offline Decryption (Offline dictionary attacks, um passwörter zu knacken)
  - erfüllt Perfect Forward Secrecy
- + Opportunistic Wireless Encryption (OWE)
  - + verschlüsselt auch offene Netzwerke, also solche, die nicht passwortgeschützt sind
  - + Verbindung ist nicht passwortbasiert, sondern geht über DH → jeder Client bekommt eigene dynamische Einmal-Schlüssel
  - bietet keine Authorization → schützt nicht vor Honeypots!
- + Verbesserte Usability durch Device Provisioning Protocol (DPP):
  - ersetzt veralteten WiFi-Protected-Setup (WPS) von WPA2
  - Gerätekonfiguration ist einfacher
  - Geräte können sich über QR-Codes oder NFC beim WLAN authentifizieren



# Datenschutz

- = Schenken der Rechte von Data Subjects über die Verarbeitung ihrer Daten
- Datenschutz = Menschenrecht
- Private Daten sind wertvoll für Unternehmen, weil sie viel Geld damit verdienen können
- Facebook Cambridge Analytica Skandal
  - Cambridge Analytica ist eine politische Beratungsfirma, die psychografische Profile von Wählern erstellt
  - Sie haben für ihre „wissenschaftliche Forschung“ eine Facebook-App entwickelt
  - Neben 270.000 zugestimmten Profilen haben sie aber auch 50 Millionen andere Facebook-Profilen ohne deren Zustimmung analysiert
  - Folge: Trump wurde Präsident
- „Facebook knows you better than your partner...“
- NSA Skandal 2013
  - geheime Dokumente wurden veröffentlicht, die massive Überwachung und Spionage aufdecken
- Social-Credit-System in China
  - alle Bürger, Unternehmen und Behörden werden zentral überwacht
  - finanzielles, soziales, moralisches und politisches Verhalten wird bewertet
- General Data Protection Regulation (GDPR)
  - einheitliche Datenschutzregelungen in der EU
  - seit 25. Mai 2018
  - Deutsche Version: Datenschutz-Grundverordnung (DSGVO) und Bundesdatenschutzgesetz (BDSG)
  - davor gab es auch schon Regelungen, aber die hat niemanden interessiert, da die Strafen nicht so schlimm waren

Na ja, ob sie das auch wirklich verdienen...?

## Was sind Persönliche Daten?

- = jede Information, die zu einer identifizierten oder identifizierbaren natürlichen Person (Data Subject) gehören
- z.B. Name, Adresse, Geburtsort, Geschlecht, ...
- Meinungen, Hobbys, Führungszeugnis/Vorstrafen, ...
- IDs, KFZ-Kennzeichen, Versicherungsinformation, ...
- Bankinformation, Einkommen, finanzielle Umstände, ...
- Standortdaten, IP-Adresse, MAC-Adresse, Cookies, ...
- z.B. auch physische, physiologische, genetische Eigenschaften, kulturelle, soziale Identität, mentale und wirtschaftliche Lage

→ Art. 9 GDPR: Höhere Sicherheitskriterien für Daten über: ethnische Herkunft, Religion, Politik, Philosophie, Genetik, Biometrie, Gesundheit, Sex

## Wann darf man personenbezogene Daten verarbeiten?

- Mit Einwilligung der betroffenen Personen
  - Einwilligung muss freiwillig, spezifisch, informiert und eindeutig sein
  - Antrags zur Einwilligung müssen klar als solche erkennbar und von anderen Inhalten getrennt sein
  - Betroffene Personen können ihre Einwilligung jederzeit widerrufen
  - Einwilligung muss dokumentiert und nachweisbar sein
- Bei Notwendigkeit der Verarbeitung gemäß DSGVO / GDPR
  - für die Erfüllung eines Vertrags, bei dem die betroffene Person beteiligt ist
  - für die Erfüllung gesetzlicher Pflichten
  - für den Schutz der lebenswichtigen Interessen der betroffenen oder einer anderen Person
  - für Aufgaben im öffentlichen Interesse oder zur Ausübung öffentlichen Gewalt
  - aus berechtigtem Interesse des Verantwortlichen oder Dritter

auch: wenn Daten öffentlich verfügbar sind

## Was sind Rechte von Data Subjects?

- Informiert sein
- Zugriffsrecht und Übertragbarkeit
- Verarbeitung einschränken + Widerspruch einlegen
- Recht auf Berichtigung und Löschung

## Was sind Pflichten von Data Processors?

- Führung eines elektronischen Verzeichnisses von Verarbeitungstätigkeiten
  - Verarbeitungszwecke
  - Kategorien von Daten und Personen
  - Datempfänger
  - Löschfristen
- Verwirklichung der Rechte der betroffenen Personen
  - Anfragen und Anliegen annehmen und bearbeiten
  - Eskalationsverfahren für komplexere Anfragen und Beschwerden
- Umgang mit Datenlecks
  - Meldepflicht an Aufsichtsbehörde und betroffene Personen innerhalb von 72h
  - Umsetzung geeigneter Gegenmaßnahmen
- Risikomanagement
  - Risikoanalysen
  - Data Protection Impact Assessment

## Konsequenzen von Datenlecks

- Bis zu 20 Mio € Strafe
- Bis zu 4% des Umsatzes
- Data Subjects haben das Recht auf Schadensersatz

# Data Protection Concept

- Für eine IT-Anwendung muss ein **Datenfeldkatalog** erstellt werden
  - alle persönlichen Datenfelder müssen zusammen mit ihrem intended use aufgelistet werden
  - WELCHE Daten?
    - Welche PERSONENGRUPPEN sind betroffen?
  - WARUM? / Welchen Zweck?
    - WIE LANGE brauche ich die Daten?
  - In welche **Data Privacy Class (DPC)** fällt das?
- DPC 0 - public:
  - öffentlich zugängliche Informationen
- DPC 1 - intern:
  - z.B. Name, E-Mail, Telefon
  - generell einsehbare Daten auf sozialen Netzwerken, zu denen nur eine geschlossene, authenticated User-Gruppe Zugang hat
  - nur firmenintern einsehbare Daten ohne zusätzliche Informationen
  - pseudonymisierte Daten** (= ich kann die Daten verknüpfen und einer Person zuordnen, aber // nicht direkt auf sie schließen)
  - anonymisierte Daten** (= ich entferne unwiderföhrlich z.B. den Namen und kann die Daten nicht mehr der Person zuordnen)
- DPC 2 - vertraulich:
  - z.B. Adresse, Geburtsdatum, Nationalität, Personalausweis, Reisepass, Anmeldeinformationen, Vhrträge, Kontostand, Angestelltenaten (Qualifikation, Stenachlasse, Fehltag, Urlaub, Krankenversicherung, ...), ...
- DPC 3 - Streng Vertraulich:
  - z.B. MAC-Adresse, IP-Adresse, aktueller (Live-) Standort, Bankkonto, sensible Daten gemüß Art. 9 GDPR (s.o.; Gesundheit, Politik, Religion, Ethnizität, Sex, Biometrische Daten, ...) sensible Angestelltenaten (Lohn, Behinderungen, biometrische Zugangsdaten, finanzielle Lage, ...)

ermöglicht große Datenmengen und KI

kann beliebig lange gespeichert und verwendet werden

## Die 7 Datenschutzprinzipien der DSGVO / GDPR

„Basic Principles der DSGVO“

- Rechtsmüßigkeit, Fairness und Transparenz
- Zweckbindung → warum brauche ich die Daten?
- Datenminimierung → welche Daten brauche ich?
- Korrektheit der Daten
- Speicherzeitbegrenzung → wie lange brauche ich die Daten?
- Integrität und Confidentialität
- Verantwortlichkeit

## Privacy-Design-Strategien und -Patterns

- Datenorientiert**
  - nur speichern, was man braucht
  - personenbezogene Daten und ihre Zusammenhänge verstecken
  - personenbezogene Daten vom Rest getrennt verarbeiten
  - personenbezogene Daten aggregiert und mit niedrigen Detailtiefe verarbeiten
- Prozessorientiert**
  - Datensubjekte ausreichend informieren
  - Datensubjekte ausreichend Kontrolle geben
  - Sicherheitsrichtlinien
  - Einhaltung der Vorschriften nachweisen
- Anonymisierung, Löschung von Metadaten
- Pseudonymisierung, Verschlüsselung, Beziehungen Löschen
- User-Data-Confinement-Pattern, Speichern für Personendaten
- Aggregation über die Zeit
- Privacy Dashboard, Privacy Icons, Datensch-Benachrichtigungen
- Benutzerzentriertes Identitätsmanagement, Ende-zu-Ende-Verschlüsselung
- Selective Access Control
- Datenschutzzmanagementssysteme, Zertifikate
- d.h. Daten beim User speichern, um Eingrenzkontrolle zu erhöhen ???

## Datensicherheit

= technische Sicherheit, Wartung und Verfügbarkeit der Datenverarbeitungssysteme und der verarbeiteten Daten

- bezieht alle Daten einer Firma
- Technical and Organizational Measures (TOM)** sind gesetzlich vorgeschrieben
  - Physische Zugriffskontrolle
  - Logische Zugriffskontrolle → Authentication + Authorization
  - User-Activity-Control → Logging
  - Verfügbarkeitskontrolle → Redundanz
  - Segregation Control → Daten, die für verschiedene Zwecke gesammelt werden, dürfen bei den Verbrüngen nicht gemischt werden
  - Verantwortlichkeiten klar im Vertrag regeln
  - Sichere Datenübertragung
  - Reviews und Bewertung
- Vertragspflicht: **Data Processing Agreement (DPA)** durch GDPR Art. 28 Abs. 3 vorgeschrieben



